differential-equations: the Differential Equations program — user guide

differential-equations-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Differential Equations covers first- and second-order ODEs, linear systems, Laplace transforms, stability and phase portraits, numerical methods, classic PDEs (heat, wave, Laplace) and dynamical systems. This guide is about `differential-equations`, the model as a self-contained program for Windows and Linux: 77 capabilities in 7 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/differential-equations-ai999x/); the Python package (`pip install differential-equations-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Differential Equations	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-differential-equations-1.4.0-windows-x86_64.zip
Desktop app: Differential Equations	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-differential-equations-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip

what	system	download
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Differential Equations in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-differential-equations-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-differential-equations.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-differential-equations-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-differential-equations-1.4.0-linux-x86_64.tar.gz
./ai999x-differential-equations-1.4.0-linux-x86_64/ai999x-differential-equations
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `differential-equations` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
differential-equations --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
differential-equations --version
```

In your browser

Open https://www.ai999x.com/models/differential-equations-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install differential-equations-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ differential-equations --version
differential-equations-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ differential-equations ask "Solve y'' + 3y' + 2y = 0 with y(0) = 1, y'(0) = 0; find y(1)."
Read as: Second-order homogeneous linear ODE: initial value problem
y = 0.6004235991
damping = overdamped
  by: Second-order homogeneous linear ODE: initial value problem  (y = e^{mt}[y0 C(t) + (v0 - m y0)
S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign of the discriminant)
  a = 1
  b = 3
  c = 2
  t = 1
  v0 = 0
  y0 = 1
  assumes: a != 0
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ differential-equations search "second-order homogeneous linear ode: initial value problem"
second_order_homogeneous_ivp         Second-order homogeneous linear ODE: initial value problem
(second-order ODEs / homogeneous)
forced_oscillator_ivp                Sinusoidally forced linear oscillator: initial value problem
(second-order ODEs / forced equations)
boundary_value_problem               Linear two-point boundary value problem (constant
coefficients)  (second-order ODEs / boundary value problems)
characteristic_roots                 Second-order constant-coefficient ODE: characteristic roots
(second-order ODEs / homogeneous)
ode_impulse_response                 Impulse response of a second-order ODE  (Laplace transforms /
impulse and step)
cauchy_euler_ivp                     Cauchy-Euler (equidimensional) equation: initial value problem
(second-order ODEs / variable coefficients)
ode_step_response                    Unit-step response of a second-order ODE  (Laplace transforms
/ impulse and step)
linear_first_order_exponential_forcing First-order linear ODE with exponential forcing  (first-
order ODEs / linear equations)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ differential-equations list
amplification_factor                    Stability function (amplification factor) of a one-step
method
autonomous_equilibria                   Equilibria of y' = f(y) and their stability (phase line)
backward_euler_method                   Backward (implicit) Euler method
bernoulli_equation                      Bernoulli equation y' + p y = q y^n
boundary_value_problem                  Linear two-point boundary value problem (constant
coefficients)
brusselator_hopf                        Brusselator: equilibrium, its type, and the Hopf value b =
1 + a^2
cauchy_euler_ivp                        Cauchy-Euler (equidimensional) equation: initial value
problem
cfl_condition                           CFL condition for the advection equation (upwind scheme)
… 69 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ differential-equations info second_order_homogeneous_ivp
Second-order homogeneous linear ODE: initial value problem
  y = e^{mt}[y0 C(t) + (v0 - m y0) S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign
of the discriminant
  in   a        coefficient of y'' [1]
  in   b        coefficient of y' [1]
  in   c        coefficient of y [1]
  in   y0       initial value y(0) [1]
  in   v0       initial slope y'(0) [1]
  in   t        time [1]
  out  y        value y(t) [1]
  out  damping  damping case []
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ differential-equations solve second_order_homogeneous_ivp a=1.0 b=3.0 c=2.0 y0=1.0 v0=0.0 t=1.0
y = 0.6004235991
damping = overdamped
  by: Second-order homogeneous linear ODE: initial value problem  (y = e^{mt}[y0 C(t) + (v0 - m y0)
S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign of the discriminant)
  a = 1
  b = 3
  c = 2
  y0 = 1
  v0 = 0
  t = 1
  assumes: a != 0
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ differential-equations solve second_order_homogeneous_ivp a=1.0 b=3.0 c=2.0 y0=1.0 v0=0.0 t=1.0 -
-json
{"outcome":"SOLVED","message":"","value":0.600423599106,"unit":"","solved_for":"y","si":[["a",1.0],
["b",3.0],["c",2.0],["y0",1.0],["v0",0.0],["t",1.0]],"values":[["y",0.600423599106,""],
["damping","overdamped",""]],"target":"y","phrase":"value y(t)","assumptions":["a !=
0"],"name":"Second-order homogeneous linear ODE: initial value problem","equation":"y = e^{mt}[y0
C(t) + (v0 - m y0) S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign of the
discriminant"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ differential-equations solve second_order_homogeneous_ivp a=1.0 b=3.0 c=2.0 y0=1.0 v0=0.0
NEEDS_INPUT: second_order_homogeneous_ivp needs t (time). Example:
solve('second_order_homogeneous_ivp', a=1, b=3, c=2, y0=1, v0=0, t=1).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ differential-equations figures second_order_homogeneous_ivp a=1.0 b=3.0 c=2.0 y0=1.0 v0=0.0 t=1.0
--out .
[
  {
    "slug": "second-order-homogeneous-ivp-curve-2",
    "title": "Value at time t against the coefficient c, for three initial values",
    "caption": "y(t) against c from 0.2 to 20 on a logarithmic axis, one curve for each initial
value y0 (1, 0.5, 2), 31 points per curve computed by the model, other inputs as entered. Marked:
the request, c = 2, y = 0.6004. Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t =
1.0.",
    "file": ".\\second-order-homogeneous-ivp-curve-2.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-1",
    "title": "Value y(t) as coefficient of y varies",
    "caption": "Value y(t) as c runs from 0.5 to 3.5, every other input as you entered it (41
solutions). Marked: your c = 2 gives y = 0.6004. Shaded by damping case: overdamped, underdamped.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-1.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-2",
    "title": "Damping case over c and b",
    "caption": "Damping case for 121 11 x 11 combinations of c and b around your inputs (the other
inputs as entered); the ring marks your inputs, which give overdamped. Blank cells admit no answer.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-2.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-3",
    "title": "Value y(t) as coefficient of y' varies",
    "caption": "Value y(t) as b runs from 0.75 to 5.25, every other input as you entered it (29
solutions). Marked: your b = 3 gives y = 0.6004. Shaded by damping case: underdamped, overdamped.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-3.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | differential-equations batch

`cases.jsonl`:

```
{"key": "second_order_homogeneous_ivp", "inputs": {"a": 1.0, "b": 3.0, "c": 2.0, "y0": 1.0, "v0":
0.0, "t": 1.0}}
{"key": "logistic_value", "inputs": {"P0": 10.0, "r": 0.5, "K": 1000.0, "t": 8.0}}
```

```
$ differential-equations batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":0.600423599106,"unit":"","solved_for":"y","si":[["a",1.0],
["b",3.0],["c",2.0],["y0",1.0],["v0",0.0],["t",1.0]],"values":[["y",0.600423599106,""],
["damping","overdamped",""]],"target":"y","phrase":"value y(t)","assumptions":["a !=
0"],"name":"Second-order homogeneous linear ODE: initial value problem","equation":"y = e^{mt}[y0
C(t) + (v0 - m y0) S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign of the
discriminant"}
{"outcome":"SOLVED","message":"","value":355.460987137,"unit":"","solved_for":"P","si":
[["P0",10.0],["r",0.5],["K",1000.0],["t",8.0]],"values":
[["P",355.460987137,""]],"target":"P","phrase":"population P(t)","assumptions":["P0 > 0","r > 0","K
> 0"],"name":"Logistic growth: population at a time","equation":"P = K / (1 + (K/P0 - 1) e^{-r
t})"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `differential-equations info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`differential-equations solve ... --json` prints one answer as one line of JSON. `differential-equations batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | differential-equations batch`. `differential-equations catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["differential-equations", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`differential-equations figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ differential-equations figures second_order_homogeneous_ivp a=1.0 b=3.0 c=2.0 y0=1.0 v0=0.0 t=1.0
--out .
[
  {
    "slug": "second-order-homogeneous-ivp-curve-2",
    "title": "Value at time t against the coefficient c, for three initial values",
    "caption": "y(t) against c from 0.2 to 20 on a logarithmic axis, one curve for each initial
value y0 (1, 0.5, 2), 31 points per curve computed by the model, other inputs as entered. Marked:
the request, c = 2, y = 0.6004. Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t =
1.0.",
    "file": ".\\second-order-homogeneous-ivp-curve-2.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-1",
    "title": "Value y(t) as coefficient of y varies",
    "caption": "Value y(t) as c runs from 0.5 to 3.5, every other input as you entered it (41
solutions). Marked: your c = 2 gives y = 0.6004. Shaded by damping case: overdamped, underdamped.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-1.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-2",
    "title": "Damping case over c and b",
    "caption": "Damping case for 121 11 x 11 combinations of c and b around your inputs (the other
inputs as entered); the ring marks your inputs, which give overdamped. Blank cells admit no answer.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-2.svg"
  },
  {
    "slug": "second-order-homogeneous-ivp-live-3",
    "title": "Value y(t) as coefficient of y' varies",
    "caption": "Value y(t) as b runs from 0.75 to 5.25, every other input as you entered it (29
solutions). Marked: your b = 3 gives y = 0.6004. Shaded by damping case: underdamped, overdamped.
Your inputs: a = 1.0, b = 3.0, c = 2.0, y0 = 1.0, v0 = 0.0, t = 1.0.",
    "file": ".\\second-order-homogeneous-ivp-live-3.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `differential_equations_ai999x.wasm` (https://www.ai999x.com/compute/rust/differential_equations_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("differential_equations_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `differential-equations info <id>` shows the same inputs and outputs in the terminal.

Classic PDES (7)

Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends — `sturm_liouville_eigenvalues`

The n-th eigenvalue and eigenfunction of y'' + lambda y = 0 on [0, L] with Dirichlet (y = 0) or Neumann (y' = 0) conditions at each end.

$$\lambda_n = \left(\frac{n\pi}{L}\right)^2 \text{(DD, NN)}, \quad \left(\frac{(n-\frac{1}{2})\pi}{L}\right)^2 \text{(mixed)}$$

symbol		meaning	SI unit
`bc`	in	boundary conditions at x = 0 and x = L	—
`L`	in	interval length (> 0)	—
`n`	in	index n >= 1 (Neumann-Neumann also has lambda_0 = 0)	—
`lambda_n`	out	n-th eigenvalue	—
`sqrt_lambda`	out	sqrt(lambda_n)	—
`eigenfunction`	out	eigenfunction	—

Applies when: L > 0; n >= 1.

```
$ differential-equations solve sturm_liouville_eigenvalues L=1 n=2 bc=dirichlet-dirichlet
lambda_n = 39.4784176
sqrt_lambda = 6.283185307
eigenfunction = sin(6.28318530718 x)
  by: Eigenvalues of y'' + lambda y = 0 with Dirichlet/Neumann ends  (sin/cos solutions; the
boundary conditions quantise sqrt(lambda))
  L = 1
  bc = dirichlet-dirichlet
  assumes: L > 0; n >= 1
```

What the worked example means: This calculation finds the specific energy levels for a vibrating string or heat distribution on an interval of length 1. Applying boundary conditions at both ends shows that the n-th mode has an eigenvalue of 39.4784. The corresponding spatial pattern, known as the eigenfunction, oscillates with a frequency proportional to 6.28319, which dictates how many half-waves fit within the domain.

Eigenvalues with a Robin (convective) end condition — `robin_eigenvalues`

The n-th eigenvalue of y'' + lambda y = 0, y(0) = 0, y'(L) + h y(L) = 0 (heat loss at one end): root of tan(mu L) = -mu/h.

$$\tan(\mu L) = -\frac{\mu}{h}, \quad \lambda = \mu^2$$

symbol		meaning	SI unit
`L`	in	interval length (> 0)	—
`h`	in	Robin coefficient (> 0)	—
`n`	in	index n >= 1	—
`lambda_n`	out	n-th eigenvalue mu_n^2	—
`sqrt_lambda`	out	mu_n	—

Applies when: L > 0; h > 0.

```
$ differential-equations solve robin_eigenvalues L=1 h=1 n=1
lambda_n = 4.115858366
sqrt_lambda = 2.028757838
  by: Eigenvalues with a Robin (convective) end condition  (bisection for mu cos(mu L) + h sin(mu
L) = 0 in ((n - 1/2) pi/L, n pi/L))
  L = 1
  h = 1
  assumes: L > 0; h > 0
```

What the worked example means: This calculation finds the nth eigenvalue for a rod of length 1 losing heat at one end with coefficient 1. The result 4.11586 corresponds to the frequency parameter 2.02876, derived from the transcendental equation governing convective boundary conditions.

Fourier sine series coefficient (half-range expansion) — `fourier_sine_coefficient`

The coefficient b_n of the sine series of f(x) on [0, L], used for the heat and wave equations with fixed ends.

$$b_n = \frac{2}{L}\int_0^L f(x)\sin\frac{n\pi x}{L}\,dx$$

symbol		meaning	SI unit
`f`	in	initial profile f(x), a polynomial	—
`L`	in	length of the interval (> 0)	—
`n`	in	index n >= 1	—
`b_n`	out	sine coefficient b_n	—

Applies when: f a polynomial; L > 0.

```
$ differential-equations solve fourier_sine_coefficient "f=x*(1 - x)" L=1 n=1
b_n = 0.2580122755
  by: Fourier sine series coefficient (half-range expansion)  (exact integration by parts of x^k
sin(n pi x / L))
  L = 1
  f = x*(1 - x)
  assumes: f a polynomial; L > 0
```

What the worked example means: This calculation determines the Fourier sine series coefficient, denoted as 0.258012, by integrating the product of the initial profile x*(1 - x) and the sine basis function over the domain defined by length 1. The result represents the amplitude of the nth mode in the half-range expansion, which is

crucial for solving boundary value problems such as the heat or wave equations with fixed ends. The computation assumes that x*(1 - x) is a polynomial and 1 is strictly positive.

Heat equation on a rod with ends at zero: Fourier series value — `heat_equation_series`

u(x, t) for u_t = alpha u_xx, u(0, t) = u(L, t) = 0, u(x, 0) = f(x) (a polynomial), summed over the first N series terms.

$$u(x,t) = \sum_{n=1}^{N} b_n e^{-\alpha (n\pi/L)^2 t} \sin \frac{n\pi x}{L}$$

symbol		meaning	SI unit
`f`	in	initial profile f(x), a polynomial	—
`L`	in	rod length (> 0)	—
`alpha`	in	diffusivity (> 0)	—
`N`	in	number of series terms (1 to 500)	—
`x`	in	position (0 <= x <= L)	—
`t`	in	time (>= 0)	—
`u`	out	temperature u(x, t) from N terms	—
`b_1`	out	first sine coefficient	—

Applies when: f a polynomial; Dirichlet ends at 0; series truncated after N terms (stated).

```
$ differential-equations solve heat_equation_series "f=x*(1 - x)" L=1 alpha=1 N=10 x=0.5 t=0.1
u = 0.09616187143
b_1 = 0.2580122755
  by: Heat equation on a rod with ends at zero: Fourier series value  (separation of variables: u =
sum b_n e^{-alpha (n pi/L)^2 t} sin(n pi x/L))
  L = 1
  alpha = 1
  x = 0.5
  t = 0.1
  f = x*(1 - x)
  assumes: f a polynomial; Dirichlet ends at 0; series truncated after N terms (stated)
```

What the worked example means: This calculation determines the temperature distribution along a rod of length 1 with diffusivity 1, given an initial polynomial profile. By applying separation of variables, the solution is expressed as a truncated Fourier sine series with 10 terms. The resulting temperature value at the specified location and time is 0.0961619, while the first coefficient in this series expansion is 0.258012. This method assumes both ends of the rod are held at zero temperature throughout the process.

Laplace's equation on a rectangle: series value — `laplace_equation_rectangle`

Steady state u_xx + u_yy = 0 on [0, a] x [0, b] with u = f(x) on the top edge y = b and 0 on the other three edges, from N terms.

$$u(x,y) = \sum_{n=1}^{N} b_n \frac{\sinh(n\pi y/a)}{\sinh(n\pi b/a)} \sin\frac{n\pi x}{a}$$

symbol		meaning	SI unit
`f`	in	top-edge values f(x), a polynomial	—
`a`	in	width (> 0)	—
`b`	in	height (> 0)	—
`N`	in	number of series terms	—
`x`	in	x in [0, a]	—
`y`	in	y in [0, b]	—
`u`	out	u(x, y) from N terms	—

Applies when: polynomial f; zero on three edges; N terms (stated).

```
$ differential-equations solve laplace_equation_rectangle f=100 a=1 b=1 N=50 x=0.5 y=0.5
u = 25
  by: Laplace's equation on a rectangle: series value  (u = sum b_n sinh(n pi y/a)/sinh(n pi b/a)
sin(n pi x/a))
  a = 1
  b = 1
  x = 0.5
  y = 0.5
  f = 100
  assumes: polynomial f; zero on three edges; N terms (stated)
```

What the worked example means: This calculation determines the potential field inside a rectangular domain with width 1 and height 1. The top edge has a specified polynomial profile, while the other three edges are held at zero. Using a truncated series of 50 terms, the resulting value at the specific coordinates x equal to 0.5 and y equal to 0.5 is calculated as 25. This represents the equilibrium state of the system under these boundary conditions.

Vibrating string with fixed ends: Fourier series value — `wave_equation_string_series`

u(x, t) for u_tt = c^2 u_xx, u(0, t) = u(L, t) = 0, u(x, 0) = f(x), u_t(x, 0) = g(x) (polynomials), from N series terms.

$$u = \sum_{n=1}^{N} \left[b_n \cos\frac{n\pi ct}{L} + \frac{B_n L}{n\pi c} \sin\frac{n\pi ct}{L} \right] \sin\frac{n\pi x}{L}$$

symbol		meaning	SI unit
`f`	in	initial profile f(x), a polynomial	—
`L`	in	string length (> 0)	—
`c`	in	wave speed (> 0)	—
`N`	in	number of series terms	—
`x`	in	position (0 <= x <= L)	—
`t`	in	time	—

symbol		meaning	SI unit
`g`	in	initial velocity g(x), a polynomial (default 0) (optional)	—
`u`	out	displacement u(x, t) from N terms	—

Applies when: polynomial f, g; fixed ends; N terms (stated).

```
$ differential-equations solve wave_equation_string_series "f=x*(1 - x)" L=1 c=2 N=20 x=0.25 t=0.3
u = -0.04999289963
  by: Vibrating string with fixed ends: Fourier series value  (u = sum [b_n cos(n pi c t/L) + B_n
L/(n pi c) sin(n pi c t/L)] sin(n pi x/L))
  L = 1
  c = 2
  x = 0.25
  t = 0.3
  f = x*(1 - x)
  assumes: polynomial f, g; fixed ends; N terms (stated)
```

What the worked example means: This calculation determines the vertical displacement of a string fixed at both ends, given its length 1, wave speed 2, and initial conditions defined by polynomials. The result uses a truncated Fourier series with 20 terms to approximate the motion, yielding a displacement value of -0.0499929 at the specified location 0.25 and moment 0.3. This method ensures that the boundary conditions are met while capturing the oscillatory behavior driven by the initial shape and velocity.

Wave equation on the whole line: d'Alembert's formula — `dalembert_solution`

u(x, t) for u_tt = c^2 u_xx on the real line with u(x, 0) = f(x), u_t(x, 0) = g(x).

$$u(x,t) = \frac{f(x-ct) + f(x+ct)}{2} + \frac{1}{2c}\int_{x-ct}^{x+ct} g(s)\,ds$$

symbol		meaning	SI unit
`f`	in	initial displacement f(x)	—
`g`	in	initial velocity g(x)	—
`c`	in	wave speed (> 0)	—
`x`	in	position	—
`t`	in	time	—
`u`	out	displacement u(x, t)	—

Applies when: c > 0; g integrable.

```
$ differential-equations solve dalembert_solution "f=sin(x)" g=0 c=2 x=1 t=0.5
u = 0.4546487134
  by: Wave equation on the whole line: d'Alembert's formula  (u = [f(x - ct) + f(x + ct)]/2 +
(1/2c) * integral of g over [x - ct, x + ct] (tanh-sinh quadrature))
  c = 2
  x = 1
  t = 0.5
  g = 0
  f = sin(x)
  assumes: c > 0; g integrable
```

What the worked example means: This result represents the wave displacement u(x, t) for the one-dimensional wave equation on an infinite domain. It is derived from d'Alembert's formula, which combines the initial displacement profile evaluated at shifted positions with the accumulated effect of the initial velocity over the characteristic interval defined by the wave speed 2. The computed value 0.454649 reflects the superposition of right-traveling and left-traveling waves at the specific location 1 and moment 0.5, assuming the medium properties remain constant.

Dynamical systems (10)

Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2 — `brusselator_hopf`

For x' = a - (b + 1) x + x^2 y, y' = b x - x^2 y: the equilibrium (a, b/a), its type and the critical b where a limit cycle is born.

$$b_c = 1 + a^2$$

symbol		meaning	SI unit
`a`	in	parameter a (> 0)	—
`b`	in	parameter b (> 0)	—
`x_eq`	out	x at equilibrium	—
`y_eq`	out	y at equilibrium	—
`b_critical`	out	Hopf bifurcation value 1 + a^2	—
`type`	out	equilibrium type	—
`stability`	out	stability	—

Applies when: a, b > 0.

```
$ differential-equations solve brusselator_hopf a=1 b=3
x_eq = 1
y_eq = 3
b_critical = 2
type = unstable spiral
stability = unstable
  by: Brusselator: equilibrium, its type, and the Hopf value b = 1 + a^2  (Jacobian [[b - 1, a^2],
[-b, -a^2]]: trace b - 1 - a^2 changes sign at the Hopf point)
  a = 1
  b = 3
  assumes: a, b > 0
```

What the worked example means: This calculation determines the steady state of the Brusselator system for given parameters a and b. The equilibrium concentrations are found to be 1 for x and 3 for y. The type of this equilibrium is classified as unstable spiral with stability status unstable. A critical threshold for the parameter b is identified at 2, marking the point where a limit cycle emerges through a Hopf bifurcation.

Competing species: coexistence equilibrium and its stability — `competing_species_equilibrium`

For x' = x(a1 - b11 x - b12 y), y' = y(a2 - b21 x - b22 y): the coexistence equilibrium and its type (stable coexistence or saddle).

$$J = \begin{pmatrix} -b_{11}x^* & -b_{12}x^* \\ -b_{21}y^* & -b_{22}y^* \end{pmatrix}$$

symbol		meaning	SI unit
`a1`	in	growth rate of x	—
`b11`	in	self-limitation of x	—
`b12`	in	effect of y on x	—
`a2`	in	growth rate of y	—
`b21`	in	effect of x on y	—
`b22`	in	self-limitation of y	—
`x_eq`	out	x at coexistence	—
`y_eq`	out	y at coexistence	—
`type`	out	equilibrium type	—
`stability`	out	stability	—

Applies when: both coexistence values positive; nullclines not parallel.

```
$ differential-equations solve competing_species_equilibrium a1=3 b11=1 b12=1 a2=2 b21=0.5 b22=1
x_eq = 2
y_eq = 1
type = stable node
stability = asymptotically stable
  by: Competing species: coexistence equilibrium and its stability  (solve the linear nullcline
equations exactly; classify the Jacobian there by trace and determinant)
  a1 = 3
  b11 = 1
  b12 = 1
  a2 = 2
  b21 = 0.5
  b22 = 1
  assumes: both coexistence values positive; nullclines not parallel
```

What the worked example means: This calculation determines the population levels where two competing species can stably coexist by solving the linear equations derived from their nullclines. The resulting equilibrium values are 2 for the first species and 1 for the second. Evaluating the Jacobian matrix at this point classifies the stability of this state, which is determined to be asymptotically stable. This indicates whether the system returns to these population levels after small disturbances or diverges away.

Logistic map: fixed points and their stability — `logistic_map_fixed_points`

Fixed points of x -> r x (1 - x) in [0, 1], their multipliers f'(x*) and stability.

$$x^* = 1 - \frac{1}{r}, \quad f'(x^*) = 2 - r$$

symbol		meaning	SI unit
`r`	in	parameter (0 < r <= 4)	—
`fixed_points`	out	fixed points in [0, 1]	—
`multipliers`	out	f'(x*) at each	—
`stability`	out	stable / unstable / neutral	—
`count`	out	number of fixed points	—

Applies when: 0 < r <= 4.

```
$ differential-equations solve logistic_map_fixed_points r=2.5
fixed_points = [0.0,0.6]
multipliers = [2.5,-0.5]
stability = ["unstable","stable"]
count = 2
  by: Logistic map: fixed points and their stability  (x* = 0 and 1 - 1/r; multipliers r and 2 - r)
  r = 2.5
  assumes: 0 < r <= 4
```

What the worked example means: This calculation determines the equilibrium states of the logistic population model for a specific growth rate. Using the input parameter 2.5, the system identifies the fixed points located at zero and one minus the reciprocal of r. The stability of each point is assessed by evaluating the derivative multiplier, which equals two minus r for the non-zero fixed point. The resulting output [unstable, stable] indicates whether nearby trajectories converge to or diverge from these equilibria, while 2 reports the total number of valid fixed points within the unit interval.

Logistic map: period-2 cycle and its stability — `logistic_map_two_cycle`

The 2-cycle {p, q} of x -> r x (1 - x) (3 < r <= 4), its multiplier f'(p) f'(q) and stability (stable for 3 < r < 1 + sqrt 6).

$$p, q = \frac{r + 1 \mp \sqrt{(r-3)(r+1)}}{2r}, \quad f'(p)f'(q) = 4 + 2r - r^2$$

symbol		meaning	SI unit
`r`	in	parameter (3 < r <= 4)	—
`p`	out	lower cycle point	—
`q`	out	upper cycle point	—
`multiplier`	out	f'(p) f'(q) = 4 + 2r - r^2	—
`stability`	out	stability	—

Applies when: 3 < r <= 4.

```
$ differential-equations solve logistic_map_two_cycle r=3.2
p = 0.5130445095
q = 0.7994554905
multiplier = 0.16
stability = stable
  by: Logistic map: period-2 cycle and its stability  (roots of f(f(x)) = x other than the fixed
points)
  r = 3.2
  assumes: 3 < r <= 4
```

What the worked example means: The period-two orbit of the logistic map is computed for parameter r = 3.2. The lower state is p = 0.513045 and the upper state is q = 0.799455. The stability is determined by the multiplier, calculated as f prime at p times f prime at q, which equals 0.16. This value indicates whether the oscillation between these two states persists or decays.

Lorenz system: equilibria and stability of the origin — `lorenz_equilibria`

Equilibria of x' = sigma(y - x), y' = x(rho - z) - y, z' = xy - beta z: the origin and C+- = (+-sqrt(beta(rho-1)), same, rho - 1).

$$C_{\pm} = \left(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1\right)$$

symbol		meaning	SI unit
`sigma`	in	Prandtl number (> 0)	—
`rho`	in	Rayleigh parameter (> 0)	—
`beta`	in	geometric factor (> 0)	—
`count`	out	number of equilibria	—
`x_c`	out	x (and y) of C+	—
`z_c`	out	z of C+- (rho - 1)	—
`origin_stability`	out	stability of the origin	—

Applies when: sigma, rho, beta > 0.

```
$ differential-equations solve lorenz_equilibria sigma=10 rho=28 beta=2.666666667
origin_stability = unstable
count = 3
x_c = 8.485281375
z_c = 27
  by: Lorenz system: equilibria and stability of the origin  (nullclines; the origin's Jacobian has
eigenvalue signs set by rho - 1)
  sigma = 10
  rho = 28
  beta = 2.666666667
  assumes: sigma, rho, beta > 0
```

What the worked example means: This calculation determines the equilibrium states of the Lorenz system using the Prandtl number, Rayleigh parameter, and geometric factor. The number of equilibria found is 3. For the symmetric non-zero equilibria, the horizontal coordinate x is 8.48528 and the vertical coordinate z is 27. The stability of the origin is classified as unstable, which depends on whether the Rayleigh parameter exceeds one.

Lorenz system: Hopf threshold rho_H (loss of stability of C+-) — `lorenz_hopf_threshold`

The Rayleigh parameter at which the equilibria C+- of the Lorenz system lose stability (subcritical Hopf; chaos follows).

$$\rho_H = \frac{\sigma(\sigma+\beta+3)}{\sigma-\beta-1}$$

symbol		meaning	SI unit
`sigma`	in	Prandtl number (> beta + 1)	—
`beta`	in	geometric factor (> 0)	—
`rho_H`	out	Hopf threshold	—

Applies when: sigma > beta + 1.

```
$ differential-equations solve lorenz_hopf_threshold sigma=10 beta=2.666666667
rho_H = 24.73684211
  by: Lorenz system: Hopf threshold rho_H (loss of stability of C+-)  (Routh-Hurwitz on the
characteristic cubic at C+-)
  sigma = 10
  beta = 2.666666667
  assumes: sigma > beta + 1
```

What the worked example means: This calculation determines the specific value of the Rayleigh parameter at which the non-zero equilibria of the Lorenz system undergo a subcritical Hopf bifurcation. Using the Prandtl number and geometric factor as inputs, the method applies Routh-Hurwitz criteria to the characteristic cubic equation. The resulting threshold, 24.7368, marks the boundary beyond which these fixed points become unstable, leading to complex dynamics and potential chaos.

Lotka-Volterra predator-prey: coexistence equilibrium and oscillation period — `lotka_volterra_equilibrium`

For x' = a x - b x y (prey), y' = -c y + d x y (predator): the coexistence equilibrium and the period of small oscillations about it.

$$(x^*, y^*) = \left(\frac{c}{d}, \frac{a}{b}\right),\ T = \frac{2\pi}{\sqrt{ac}}$$

symbol		meaning	SI unit
`a`	in	prey growth rate (> 0)	—
`b`	in	predation rate (> 0)	—
`c`	in	predator death rate (> 0)	—

symbol		meaning	SI unit
`d`	in	predator efficiency (> 0)	—
`x_eq`	out	prey at equilibrium c/d	—
`y_eq`	out	predators at equilibrium a/b	—
`omega`	out	small-oscillation angular frequency sqrt(ac)	—
`period`	out	small-oscillation period 2 pi / sqrt(ac)	—

Applies when: a, b, c, d > 0.

```
$ differential-equations solve lotka_volterra_equilibrium a=1 b=0.1 c=1.5 d=0.075
x_eq = 20
y_eq = 10
omega = 1.224744871
period = 5.130199321
  by: Lotka-Volterra predator-prey: coexistence equilibrium and oscillation period  (equilibrium of
the nullclines; the Jacobian there has eigenvalues +- i sqrt(ac))
  a = 1
  b = 0.1
  c = 1.5
  d = 0.075
  assumes: a, b, c, d > 0
```

What the worked example means: This calculation determines the steady state where predator and prey populations balance each other out. Given positive growth, predation, death, and efficiency rates, the equilibrium number of prey is found by dividing the predator death rate by the predator efficiency, resulting in 20. The equilibrium number of predators is the prey growth rate divided by the predation rate, yielding 10. Around this balance point, the system exhibits cyclic behavior. The angular frequency of these small oscillations is 1.22474, which leads to a full cycle duration of 5.1302. This period represents the time it takes for the population dynamics to complete one typical rise and fall cycle near the equilibrium.

Lyapunov exponent of the logistic map (periodic regime) — `logistic_map_lyapunov`

The Lyapunov exponent of x -> r x (1 - x): the average of ln|f'(x_k)| over n iterations after the orbit settles (fixed point or 2-cycle).

$$\lambda = \frac{1}{n}\sum_{k=1}^{n} \ln\left|r(1-2x_k)\right|$$

symbol		meaning	SI unit
`r`	in	parameter (0 < r <= 4)	—
`x0`	in	initial value in (0, 1)	—
`n`	in	iterations averaged (even)	—
`lyapunov`	out	Lyapunov exponent (negative: stable periodic orbit)	—
`attractor`	out	attractor reached	—

Applies when: the orbit settles on a fixed point or 2-cycle (|x_{k+2} - x_k| < 1e-14); n even.

```
$ differential-equations solve logistic_map_lyapunov r=2.5 x0=0.3 n=1000
lyapunov = -0.6931471806
attractor = fixed point
  by: Lyapunov exponent of the logistic map (periodic regime)  (iterate until settled, then average
ln|r (1 - 2 x_k)| over n iterations)
  r = 2.5
  x0 = 0.3
  assumes: the orbit settles on a fixed point or 2-cycle (|x_{k+2} - x_k| < 1e-14); n even
```

What the worked example means: This calculation determines the Lyapunov exponent for the logistic map using parameter 2.5 and initial value 0.3. The method iterates the map until the orbit settles, then averages the natural logarithm of the absolute derivative over 1000 steps. The resulting value of -0.693147 quantifies the average rate at which nearby trajectories converge or diverge. A negative value signifies that the system is stable and attracted to the cycle identified as fixed point.

Nonlinear 2-D system: type of an equilibrium by linearization — `nonlinear_equilibrium_type`

For x' = f(x, y), y' = g(x, y) and an equilibrium (x0, y0): the Jacobian there, the equilibrium type and stability (Hartman-Grobman).

$$J = \begin{pmatrix} f_x & f_y \\ g_x & g_y \end{pmatrix}_{(x_0, y_0)}$$

symbol		meaning	SI unit
`f`	in	x' = f(x, y)	—
`g`	in	y' = g(x, y)	—
`x0`	in	equilibrium x	—
`y0`	in	equilibrium y	—
`type`	out	type of the linearization	—
`stability`	out	stability (inconclusive for a center or a degenerate case)	—
`jacobian`	out	Jacobian at the equilibrium	—
`trace`	out	trace of the Jacobian	—
`det`	out	determinant of the Jacobian	—

Applies when: (x0, y0) is an equilibrium.

```
$ differential-equations solve nonlinear_equilibrium_type f=y "g=-sin(x) - 0.5*y" x0=0 y0=0
type = stable spiral
stability = asymptotically stable
jacobian = [[0.0,1.0],[-1.0,-0.5]]
trace = -0.5
det = 1
  by: Nonlinear 2-D system: type of an equilibrium by linearization  (exact symbolic Jacobian at
the point; trace-determinant classification)
  x0 = 0
  y0 = 0
  f = y
  g = -sin(x) - 0.5*y
  assumes: (x0, y0) is an equilibrium
```

What the worked example means: For the system defined by y and -sin(x) - 0.5*y, the Jacobian matrix is evaluated at the equilibrium point 0, 0. The resulting matrix has a trace of -0.5 and a determinant of 1. Based on these values, the linearized system exhibits behavior characteristic of a stable spiral. Consequently, the nonlinear system near this point is deemed asymptotically stable, provided the equilibrium is hyperbolic.

SIR epidemic: R0, peak infections and final size — `sir_epidemic`

For S' = -beta S I, I' = beta S I - gamma I (fractions of the population): R0, the peak infected fraction and the final susceptible fraction.

$$I_{\max} = S_0 + I_0 - \frac{1 + \ln(R_0 S_0)}{R_0}, \quad \ln\frac{S_\infty}{S_0} = R_0(S_\infty - S_0 - I_0)$$

symbol		meaning	SI unit
`beta`	in	transmission rate (> 0)	—
`gamma`	in	recovery rate (> 0)	—
`S0`	in	initial susceptible fraction	—
`I0`	in	initial infected fraction	—
`R0`	out	basic reproduction number beta/gamma	—
`I_max`	out	peak infected fraction	—
`S_inf`	out	final susceptible fraction	—
`attack_rate`	out	fraction infected during the epidemic S0 - S_inf	—

Applies when: S0, I0 > 0; S0 + I0 <= 1.

```
$ differential-equations solve sir_epidemic beta=0.3 gamma=0.1 S0=0.99 I0=0.01
R0 = 3
I_max = 0.3038126824
S_inf = 0.0587973648
attack_rate = 0.9312026352
  by: SIR epidemic: R0, peak infections and final size  (first integrals: I + S - ln(S)/R0 is
conserved; peak at S = 1/R0; final size from ln(S/S0) = R0 (S - S0 - I0))
  beta = 0.3
  gamma = 0.1
  S0 = 0.99
  I0 = 0.01
  assumes: S0, I0 > 0; S0 + I0 <= 1
```

What the worked example means: This calculation determines key metrics for a Susceptible-Infected-Recovered epidemic model using initial transmission and recovery rates along with starting population fractions. The basic reproduction number, which indicates how many secondary infections one case causes in a fully susceptible population, is computed as 3. Using conservation laws, the maximum proportion of the population that becomes infected simultaneously is found to be 0.303813. Finally, the model solves for the remaining susceptible fraction after the epidemic subsides, yielding 0.0587974, from which the total attack rate is derived.

First-order ODES (24)

Age from the fraction remaining (radioactive dating) — `radiometric_age`

The elapsed time when a fraction p of a radioactive substance with half-life T remains.

$$t = T\,\frac{\ln(1/p)}{\ln 2}$$

symbol		meaning	SI unit
`T`	in	half-life (> 0)	—
`p`	in	fraction remaining, 0 < p <= 1	—
`t`	out	age (elapsed time)	—

Applies when: T > 0; 0 < p <= 1.

```
$ differential-equations solve radiometric_age T=5730 p=0.25
t = 11460
  by: Age from the fraction remaining (radioactive dating)  (t = T ln(1/p) / ln 2)
  T = 5730
  p = 0.25
  assumes: T > 0; 0 < p <= 1
```

What the worked example means: This calculation determines the age of a sample by measuring how much radioactive material remains. Given a half-life of 5730 and a remaining fraction of 0.25, the model computes that the elapsed time is 11460. This result indicates how long ago the substance was in its initial state, assuming standard exponential decay behavior.

Bernoulli equation y' + p y = q y^n — `bernoulli_equation`

Solve the Bernoulli equation with constant p, q by the substitution v = y^{1-n}; give y(t).

$$y^{1-n} = \frac{q}{p} + \left(y_0^{1-n} - \frac{q}{p}\right) e^{-(1-n)pt}$$

symbol		meaning	SI unit
`p`	in	coefficient of y	—
`q`	in	coefficient of y^n	—
`n`	in	exponent (n != 1)	—
`y0`	in	initial value (> 0)	—
`t`	in	time (>= 0)	—
`y`	out	value y(t)	—

Applies when: p != 0; n != 1; y0 > 0; solution stays positive.

```
$ differential-equations solve bernoulli_equation p=1 q=1 n=2 y0=0.5 t=1
y = 0.2689414214
  by: Bernoulli equation y' + p y = q y^n  (v = y^{1-n}: v' + (1-n) p v = (1-n) q, linear)
  p = 1
  q = 1
  n = 2
  y0 = 0.5
  t = 1
  assumes: p != 0; n != 1; y0 > 0; solution stays positive
```

What the worked example means: This calculation determines the value of y(t) for a Bernoulli differential equation using the substitution method. Given the initial condition y0 = 0.5, coefficient p = 1, coefficient q = 1, and exponent n = 2, the model computes the state at time t = 1. The resulting value is 0.268941. This approach converts the nonlinear problem into a linear one, allowing for an explicit solution that describes how the system evolves over time under these specific constant parameters.

Doubling time or half-life from a rate constant — `doubling_time`

For dy/dt = k*y: the doubling time (k > 0) or half-life (k < 0).

$$t_2 = \frac{\ln 2}{|k|}$$

symbol		meaning	SI unit
`k`	in	rate constant	—
`t_char`	out	doubling time or half-life ln 2 / |k|	—
`kind`	out	doubling time or half-life	—

Applies when: k != 0.

```
$ differential-equations solve doubling_time k=0.07
t_char = 9.902102579
kind = doubling time
  by: Doubling time or half-life from a rate constant  (solve e^{k t} = 2 (or 1/2): t = ln 2 / |k|)
  k = 0.07
  assumes: k != 0
```

What the worked example means: This calculation determines the doubling time or half-life from the rate constant. Given a rate constant of 0.07, the resulting characteristic time is 9.9021. This value represents the duration required for the quantity to double if the rate is positive, or to halve if the rate is negative.

Draining tank (Torricelli's law): height and emptying time — `torricelli_draining`

Solve dh/dt = -k sqrt(h), h(0) = h0: water height at time t and the time to empty.

$$h(t) = \left(\sqrt{h_0} - \tfrac{kt}{2}\right)^2,\ t_{empty} = \frac{2\sqrt{h_0}}{k}$$

symbol		meaning	SI unit
`h0`	in	initial height (> 0)	—
`k`	in	drain constant (> 0)	—
`t`	in	time (>= 0)	—
`h`	out	height h(t) (0 once empty)	—
`t_empty`	out	time to empty 2 sqrt(h0)/k	—

Applies when: h0 > 0; k > 0.

```
$ differential-equations solve torricelli_draining h0=4 k=0.2 t=5
h = 2.25
t_empty = 20
  by: Draining tank (Torricelli's law): height and emptying time  (separable: sqrt(h) = sqrt(h0) -
k t / 2)
  h0 = 4
  k = 0.2
  t = 5
  assumes: h0 > 0; k > 0
```

What the worked example means: This calculation determines the remaining water height in a draining tank at a specific moment. Starting from an initial height of 4 and using a drain constant of 0.2, the model predicts that at time 5, the height will be 2.25. It also computes the total duration required for the tank to completely empty, which is 20. These values rely on Torricelli's law, assuming the flow rate is proportional to the square root of the current water height.

Equation y' = k y^2: solution and blow-up time — `quadratic_ode_blowup`

Solve y' = k y^2, y(0) = y0; give y(t) and the finite blow-up time 1/(k y0) when k y0 > 0.

$$y(t) = \frac{y_0}{1 - k y_0 t}$$

symbol		meaning	SI unit
`k`	in	coefficient	—
`y0`	in	initial value	—
`t`	in	time (>= 0)	—
`y`	out	value y(t)	—
`blowup_time`	out	time at which y becomes infinite (k y0 > 0)	—

Applies when: t before the blow-up time.

```
$ differential-equations solve quadratic_ode_blowup k=1 y0=0.5 t=1
y = 1
blowup_time = 2
  by: Equation y' = k y^2: solution and blow-up time  (separate: -1/y = k t - 1/y0)
  k = 1
  y0 = 0.5
  t = 1
  assumes: t before the blow-up time
```

What the worked example means: This calculation determines the value of the function at a specific time using the formula for this type of growth. The result, 1, represents the state after evolving from the initial condition 0.5 with coefficient 1 over duration 1. Additionally, it identifies the critical moment when the value becomes infinite, known as the blow-up time, which is calculated as 2. This time is valid only if the product of the coefficient and initial value is positive.

Equilibria of y' = f(y) and their stability (phase line) — `autonomous_equilibria`

All equilibrium solutions of an autonomous equation y' = f(y) with polynomial f and their stability from the sign of f.

$$f(y^*) = 0;\ f > 0 \text{ left},\ f < 0 \text{ right} \Rightarrow \text{stable}$$

symbol		meaning	SI unit
`f`	in	right-hand side f(y), a polynomial	—
`equilibria`	out	equilibrium values (ascending)	—
`stability`	out	stable / unstable / semi-stable for each	—
`count`	out	number of equilibria	—

Applies when: f a polynomial in y with rational coefficients, degree <= 8.

```
$ differential-equations solve autonomous_equilibria "f=y*(1 - y)"
equilibria = [0.0,1.0]
stability = ["unstable","stable"]
count = 2
  by: Equilibria of y' = f(y) and their stability (phase line)  (exact real roots of f; the sign of
f between consecutive roots (phase line))
  f = y*(1 - y)
  assumes: f a polynomial in y with rational coefficients, degree <= 8
```

What the worked example means: This calculation identifies the constant solutions for the autonomous differential equation defined by the polynomial function f(y). Finding the roots where f(y) equals zero gives the equilibrium points listed in ascending order as [0, 1]. The stability of each point is classified by examining the sign of f(y) in the intervals between these roots. A positive sign to the left and negative to the right indicates a stable equilibrium, while the opposite suggests instability. The final output provides the specific classification for each root in [unstable, stable] and the total number of real equilibria found, which is 2. This method allows for a qualitative understanding of the system's long-term behavior without explicitly solving the differential equation.

Exponential growth or decay: value at a time — `exponential_growth_value`

Solve dy/dt = k*y with y(0) = y0 and give y(t).

$$y(t) = y_0 e^{kt}$$

symbol		meaning	SI unit
`y0`	in	initial value y(0)	—
`k`	in	growth rate (negative for decay)	—
`t`	in	time	—
`y`	out	value y(t)	—

Applies when: k > 0 growth, k < 0 decay.

```
$ differential-equations solve exponential_growth_value y0=100 k=0.03 t=10
y = 134.9858808
  by: Exponential growth or decay: value at a time  (separation of variables: y = y0*exp(k*t))
  y0 = 100
  k = 0.03
  t = 10
  assumes: k > 0 growth, k < 0 decay
```

What the worked example means: This calculation determines the quantity's size at a specific moment by applying exponential growth or decay principles. It uses the initial value 100 and the constant rate parameter 0.03 to compute the result 134.986 after a duration of 10. If the rate is positive, the system grows; if negative, it shrinks. The final figure represents the accumulated change relative to the starting point under these proportional conditions.

Exponential model through two measurements — `exponential_from_two_points`

The rate k and initial value y0 of y = y0*e^{k t} through (t1, y1) and (t2, y2).

$$k = \frac{\ln(y_2/y_1)}{t_2 - t_1},\ y_0 = y_1 e^{-kt_1}$$

symbol		meaning	SI unit
`t1`	in	first time	—
`y1`	in	value at t1	—

symbol		meaning	SI unit
`t2`	in	second time	—
`y2`	in	value at t2	—
`k`	out	rate constant	—
`y0`	out	value at t = 0	—

Applies when: t1 != t2; y1, y2 nonzero, same sign.

```
$ differential-equations solve exponential_from_two_points y1=100 t1=2 y2=300 t2=5
k = 0.3662040962
y0 = 48.07498568
  by: Exponential model through two measurements  (k = ln(y2/y1)/(t2 - t1), y0 = y1 e^{-k t1})
  y1 = 100
  t1 = 2
  y2 = 300
  t2 = 5
  assumes: t1 != t2; y1, y2 nonzero, same sign
```

What the worked example means: This calculation determines the parameters of an exponential model that passes exactly through two measured data points. Using the first time 2 with value 100 and the second time 5 with value 300, it computes the growth or decay rate 0.366204 and the theoretical starting value at zero time, 48.075. These outputs allow you to predict the system's behavior at any other time based on the assumption of pure exponential change.

Falling body with linear drag: velocity — `linear_drag_velocity`

Solve m v' = m g - c v with v(0) = v0: velocity at time t and terminal velocity m g / c.

$$v(t) = \frac{mg}{c} + \left(v_0 - \frac{mg}{c}\right)e^{-ct/m}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`c`	in	linear drag coefficient (> 0)	—
`g`	in	gravitational acceleration	—
`v0`	in	initial velocity (downward positive)	—
`t`	in	time	—
`v`	out	velocity v(t)	—
`v_terminal`	out	terminal velocity m g / c	—

Applies when: m > 0; c > 0.

```
$ differential-equations solve linear_drag_velocity m=2 c=0.5 g=9.81 v0=0 t=3
v = 20.70433647
v_terminal = 39.24
  by: Falling body with linear drag: velocity  (linear first-order ODE: v = mg/c + (v0 - mg/c) e^{-
ct/m})
  m = 2
  c = 0.5
  g = 9.81
  v0 = 0
  t = 3
  assumes: m > 0; c > 0
```

What the worked example means: This calculation determines the velocity of a falling body at time 3, given its mass 2, drag coefficient 0.5, gravitational acceleration 9.81, and initial velocity 0. The result shows that the current velocity is 20.7043, which approaches the terminal velocity of 39.24 as time increases. This value represents the speed where the downward force of gravity balances the upward drag force.

Falling body with quadratic drag: velocity — `quadratic_drag_velocity`

Solve m v' = m g - c v^2 with v(0) = v0 >= 0: velocity at time t and terminal velocity sqrt(m g / c).

$$v(t) = v_t \tanh\left(\frac{gt}{v_t} + \operatorname{artanh}\frac{v_0}{v_t}\right),\ v_t = \sqrt{mg/c}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`c`	in	quadratic drag coefficient (> 0)	—
`g`	in	gravitational acceleration (> 0)	—
`v0`	in	initial downward velocity (>= 0)	—
`t`	in	time (>= 0)	—
`v`	out	velocity v(t)	—
`v_terminal`	out	terminal velocity	—

Applies when: m, c, g > 0; v0 >= 0.

```
$ differential-equations solve quadratic_drag_velocity m=80 c=0.25 g=9.81 v0=0 t=5
v = 39.45143246
v_terminal = 56.02856414
  by: Falling body with quadratic drag: velocity  (separable: v = v_t tanh(g t / v_t +
artanh(v0/v_t)) (coth form when v0 > v_t))
  m = 80
  c = 0.25
  g = 9.81
  v0 = 0
  t = 5
  assumes: m, c, g > 0; v0 >= 0
```

What the worked example means: This calculation determines the downward velocity of a falling object subject to quadratic air resistance. Given a mass of 80, a drag coefficient of 0.25, and gravitational acceleration

of 9.81, the model solves the differential equation for motion starting with an initial velocity of 0. The result, 39.4514, represents the instantaneous speed at time 5, while 56.0286 indicates the maximum constant speed the object will eventually reach as drag balances gravity.

First-order linear ODE with constant coefficients (integrating factor) — `linear_first_order`

Solve y' + a y = b with y(0) = y0 and give y(t) and the equilibrium b/a.

$$y(t) = \frac{b}{a} + \left(y_0 - \frac{b}{a}\right)e^{-at}$$

symbol		meaning	SI unit
`a`	in	coefficient of y	—
`b`	in	constant forcing	—
`y0`	in	initial value	—
`t`	in	time	—
`y`	out	value y(t)	—
`y_steady`	out	equilibrium b/a (the limit when a > 0)	—

Applies when: a != 0.

```
$ differential-equations solve linear_first_order a=2 b=6 y0=1 t=1
y = 2.729329434
y_steady = 3
  by: First-order linear ODE with constant coefficients (integrating factor)  (integrating factor e^{a t}: y = b/a + (y0 - b/a) e^{-a t})
  a = 2
  b = 6
  y0 = 1
  t = 1
  assumes: a != 0
```

What the worked example means: This calculation determines the value of the dependent variable y at a specific time t using the integrating factor method for a first-order linear ordinary differential equation. The result, 2.72933, reflects the system's evolution from its initial condition 1 toward the equilibrium value 3, which is determined by the ratio of the constant forcing b to the coefficient a.

First-order linear ODE with exponential forcing — `linear_first_order_exponential_forcing`

Solve y' + a y = b e^{c t}, y(0) = y0, and give y(t) (including the resonant case c = -a).

$$y = \frac{b}{a+c}e^{ct} + \left(y_0 - \frac{b}{a+c}\right)e^{-at}$$

symbol		meaning	SI unit
`a`	in	coefficient of y	—
`b`	in	forcing amplitude	—

symbol		meaning	SI unit
`c`	in	forcing exponent	—
`y0`	in	initial value	—
`t`	in	time	—
`y`	out	value y(t)	—
`resonant`	out	c = -a (secular term t e^{-a t})	—

Applies when: any real a, b, c.

```
$ differential-equations solve linear_first_order_exponential_forcing a=1 b=2 c=0.5 y0=0 t=2
y = 3.443928727
resonant = false
  by: First-order linear ODE with exponential forcing  (integrating factor: y = b/(a+c) e^{ct} +
(y0 - b/(a+c)) e^{-at}; c = -a: (y0 + b t) e^{-at})
  a = 1
  b = 2
  c = 0.5
  y0 = 0
  t = 2
  assumes: any real a, b, c
```

In the worked example, a = 1, b = 2, c = 0.5, y0 = 0, t = 2 go through the equation above and give y = 3.443928727, resonant = False. It applies only when any real a, b, c: outside that it says what is wrong instead of returning a number.

Gompertz growth: value at a time — `gompertz_growth`

Solve dy/dt = r y ln(K/y), y(0) = y0 (tumour and population growth) and give y(t).

$$y(t) = K \exp\left(\ln \frac{y_0}{K}\, e^{-rt}\right)$$

symbol		meaning	SI unit
`y0`	in	initial value (> 0)	—
`r`	in	growth rate	—
`K`	in	carrying capacity (> 0)	—
`t`	in	time	—
`y`	out	value y(t)	—

Applies when: y0 > 0; K > 0.

```
$ differential-equations solve gompertz_growth y0=10 r=0.3 K=1000 t=5
y = 357.881855
  by: Gompertz growth: value at a time  (u = ln(y/K) satisfies u' = -r u)
  y0 = 10
  r = 0.3
  K = 1000
  t = 5
  assumes: y0 > 0; K > 0
```

What the worked example means: This calculation determines the population or tumor size y at a specific time t, starting from an initial value y0. It uses the Gompertz growth model where growth slows as the size approaches the carrying capacity K. The result shows that with a growth rate r, the value reaches 357.882 after time t.

Logistic growth with constant harvesting: equilibria — `logistic_harvesting`

Equilibria of dy/dt = r y (1 - y/K) - H, their stability, and the critical (maximum sustainable) harvest.

$$y_{\pm} = \frac{K}{2}\left(1 \pm \sqrt{1 - \frac{4H}{rK}}\right)$$

symbol		meaning	SI unit
`r`	in	growth rate (> 0)	—
`K`	in	carrying capacity (> 0)	—
`H`	in	harvest rate (>= 0)	—
`y_high`	out	upper equilibrium (stable)	—
`y_low`	out	lower equilibrium (unstable threshold)	—
`H_critical`	out	maximum sustainable harvest rK/4	—
`stability_high`	out	stability	—
`stability_low`	out	stability	—

Applies when: r, K > 0; 0 <= H <= rK/4 (otherwise the population collapses).

```
$ differential-equations solve logistic_harvesting r=0.5 K=1000 H=100
y_high = 723.6067978
y_low = 276.3932023
H_critical = 125
stability_high = stable
stability_low = unstable
  by: Logistic growth with constant harvesting: equilibria  (roots of r y (1 - y/K) = H; sign of f'
at each)
  r = 0.5
  K = 1000
  H = 100
  assumes: r, K > 0; 0 <= H <= rK/4 (otherwise the population collapses)
```

What the worked example means: This calculation finds the population levels where growth balances constant harvesting. The upper stable equilibrium is 723.607, meaning the population tends toward this size if it starts above the lower unstable equilibrium of 276.393. The critical harvest rate, which is the maximum amount that can be taken without causing extinction, is 125. If the actual harvest exceeds this value, the population will collapse to zero.

Logistic growth: inflection time and maximum growth rate — `logistic_inflection`

When a logistic population grows fastest: the inflection time, the population there (K/2) and the maximum rate rK/4.

$$t^* = \frac{1}{r}\ln\left(\frac{K}{P_0} - 1\right),\ \dot{P}_{\max} = \frac{rK}{4}$$

symbol		meaning	SI unit
`P0`	in	initial population (0 < P0 < K/2)	—
`r`	in	growth rate (> 0)	—
`K`	in	carrying capacity	—
`t_inflection`	out	time of the inflection point	—
`P_inflection`	out	population at the inflection (K/2)	—
`max_rate`	out	largest growth rate dP/dt (rK/4)	—

Applies when: r > 0; 0 < P0 < K/2.

```
$ differential-equations solve logistic_inflection P0=10 r=0.5 K=1000
t_inflection = 9.1902397
P_inflection = 500
max_rate = 125
  by: Logistic growth: inflection time and maximum growth rate  (P'' = 0 at P = K/2; t* = ln(K/P0 -
1)/r)
  P0 = 10
  r = 0.5
  K = 1000
  assumes: r > 0; 0 < P0 < K/2
```

What the worked example means: This calculation determines the inflection point of a logistic growth curve, identifying when the population accelerates most rapidly before deceleration begins. Using the initial population 10, growth rate 0.5, and carrying capacity 1000 shows that the peak growth occurs at time 9.19024. At this specific moment, the population size is exactly half the carrying capacity, or 500, and the rate of change in population is 125. This result assumes standard logistic dynamics where the starting population is below half the carrying capacity.

Logistic growth: population at a time — `logistic_value`

Solve dP/dt = r P (1 - P/K), P(0) = P0, and give P(t).

$$P(t) = \frac{K}{1 + \left(\frac{K}{P_0} - 1\right)e^{-rt}}$$

symbol		meaning	SI unit
`P0`	in	initial population (> 0)	—
`r`	in	intrinsic growth rate (> 0)	—
`K`	in	carrying capacity (> 0)	—
`t`	in	time	—
`P`	out	population P(t)	—

Applies when: P0 > 0; r > 0; K > 0.

```
$ differential-equations solve logistic_value P0=10 r=0.5 K=1000 t=8
P = 355.4609871
  by: Logistic growth: population at a time  (P = K / (1 + (K/P0 - 1) e^{-r t}))
  P0 = 10
  r = 0.5
  K = 1000
  t = 8
  assumes: P0 > 0; r > 0; K > 0
```

What the worked example means: This calculation determines the population size at a specific time using the logistic growth model. It takes the initial population 10, the intrinsic growth rate 0.5, and the carrying capacity 1000 to solve for the current population 355.461. The result represents how many individuals exist in the system after time 8 has passed, assuming resources are limited and growth slows as the population nears the maximum sustainable level.

Mixing tank (brine): amount of salt at a time — `mixing_tank`

A well-stirred tank of volume V with inflow = outflow rate r at concentration c_in: amount A(t) of solute.

$$A(t) = c_{in}V + (A_0 - c_{in}V)e^{-rt/V}$$

symbol		meaning	SI unit
`V`	in	volume (constant, > 0)	—
`r`	in	flow rate in = out (> 0)	—
`c_in`	in	inflow concentration (>= 0)	—
`A0`	in	initial amount of solute	—
`t`	in	time	—
`A`	out	amount A(t)	—
`A_limit`	out	long-run amount c_in V	—
`concentration`	out	concentration A(t)/V	—

Applies when: V > 0; r > 0.

```
$ differential-equations solve mixing_tank V=100 r=5 c_in=0.2 A0=0 t=10
A = 7.869386806
A_limit = 20
concentration = 0.07869386806
  by: Mixing tank (brine): amount of salt at a time  (A' = r c_in - (r/V) A: A = c_in V + (A0 -
c_in V) e^{-r t/V})
  V = 100
  r = 5
  c_in = 0.2
  A0 = 0
  t = 10
  assumes: V > 0; r > 0
```

What the worked example means: This calculation determines the total mass of solute, denoted as 7.86939, present in a well-stirred tank of constant volume 100 after a duration of 10. The result is derived from the initial amount of solute 0, the constant inflow concentration 0.2, and the flow rate 5. As time progresses, the

system moves toward an equilibrium state where the amount of salt stabilizes near the long-run value 20, which represents the product of the tank volume and the inflow concentration.

Newton's law of cooling: cooling constant from a measurement — `newton_cooling_constant`

The cooling constant k when a body at T0 in surroundings Ta reads T1 after time t1.

$$k = \frac{1}{t_1} \ln \frac{T_0 - T_a}{T_1 - T_a}$$

symbol		meaning	SI unit
`T0`	in	initial temperature	—
`Ta`	in	ambient temperature	—
`T1`	in	measured temperature	—
`t1`	in	time of the measurement (> 0)	—
`k`	out	cooling constant	—

Applies when: t1 > 0; T1 strictly between Ta and T0.

```
$ differential-equations solve newton_cooling_constant T0=90 Ta=20 T1=60 t1=5
k = 0.1119231576
  by: Newton's law of cooling: cooling constant from a measurement  (k = ln((T0 - Ta)/(T1 -
Ta))/t1)
  T0 = 90
  Ta = 20
  T1 = 60
  t1 = 5
  assumes: t1 > 0; T1 strictly between Ta and T0
```

What the worked example means: This calculation determines the cooling constant k for an object using Newton's law of cooling. It uses the initial temperature 90, the ambient temperature 20, the temperature read at a later time 60, and the elapsed time 5. The result, 0.111923, quantifies how rapidly the object approaches the surrounding temperature, allowing prediction of future thermal states.

Newton's law of cooling: temperature at a time — `newton_cooling_temperature`

Solve dT/dt = -k (T - Ta), T(0) = T0, and give T(t).

$$T(t) = T_a + (T_0 - T_a)e^{-kt}$$

symbol		meaning	SI unit
`T0`	in	initial temperature	—
`Ta`	in	ambient temperature	—
`k`	in	cooling constant (> 0)	—
`t`	in	time	—
`T`	out	temperature T(t)	—

Applies when: k > 0.

```
$ differential-equations solve newton_cooling_temperature T0=90 Ta=20 k=0.1 t=5
T = 62.45714618
  by: Newton's law of cooling: temperature at a time  (T = Ta + (T0 - Ta) e^{-k t})
  T0 = 90
  Ta = 20
  k = 0.1
  t = 5
  assumes: k > 0
```

What the worked example means: This calculation determines the object's temperature at a specific moment using Newton's Law of Cooling. It combines the initial temperature 90, the constant ambient temperature 20, and the cooling rate 0.1 to predict how heat dissipates over time 5. The resulting temperature 62.4571 indicates the object's thermal state relative to its surroundings, showing that it moves closer to the ambient level as time progresses.

Newton's law of cooling: time to reach a temperature — `newton_cooling_time`

With dT/dt = -k (T - Ta), T(0) = T0: when does the temperature reach T1?

$$t = \frac{1}{k} \ln \frac{T_0 - T_a}{T_1 - T_a}$$

symbol		meaning	SI unit
`T0`	in	initial temperature	—
`Ta`	in	ambient temperature	—
`k`	in	cooling constant (> 0)	—
`T1`	in	target temperature (between Ta and T0)	—
`t`	out	time to reach T1	—

Applies when: k > 0; T1 between Ta and T0.

```
$ differential-equations solve newton_cooling_time T1=40 T0=90 Ta=20 k=0.1
t = 12.52762969
  by: Newton's law of cooling: time to reach a temperature  (t = ln((T0 - Ta)/(T1 - Ta))/k)
  T1 = 40
  T0 = 90
  Ta = 20
  k = 0.1
  assumes: k > 0; T1 between Ta and T0
```

What the worked example means: This calculation determines the duration needed for an object to cool or warm from its starting temperature of 90 to a target temperature of 40, given that the surrounding environment maintains a constant ambient temperature of 20. The result relies on a cooling constant of 0.1, which dictates the rate of heat transfer. By applying Newton's law of cooling, the model computes that the specific time elapsed is 12.5276. This outcome assumes the process follows standard exponential decay principles where the temperature difference drives the change over time.

Radioactive decay from a half-life — `half_life_decay`

Amount left after time t when dN/dt = -k*N and the half-life is T; also the decay constant k.

$$N(t) = N_0\, 2^{-t/T}, \quad k = \frac{\ln 2}{T}$$

symbol		meaning	SI unit
`N0`	in	initial amount	—
`T`	in	half-life (> 0)	—
`t`	in	elapsed time	—
`N`	out	amount remaining N(t)	—
`k`	out	decay constant ln 2 / T	—

Applies when: T > 0.

```
$ differential-equations solve half_life_decay N0=80 T=3 t=9
N = 10
k = 0.2310490602
  by: Radioactive decay from a half-life  (N = N0 * 2^(-t/T), k = ln 2 / T)
  N0 = 80
  T = 3
  t = 9
  assumes: T > 0
```

What the worked example means: This calculation determines the quantity of a radioactive substance remaining after a specific elapsed time, given its initial amount and half-life. Using the exponential decay model, the result shows that starting with 80, after 9 has passed with a half-life of 3, the remaining amount is 10. Additionally, the decay constant, which represents the probability of decay per unit time, is calculated as 0.231049.

Separable equation y' = a t^n y — `separable_power`

Solve the separable equation y' = a t^n y with y(0) = y0 and give y(t).

$$y(t) = y_0 \exp\left(\frac{a\, t^{n+1}}{n+1}\right)$$

symbol		meaning	SI unit
`a`	in	coefficient	—
`n`	in	power of t (>= 0)	—
`y0`	in	initial value	—
`t`	in	time (>= 0)	—
`y`	out	value y(t)	—

Applies when: n >= 0; t >= 0.

```
$ differential-equations solve separable_power a=2 n=1 y0=3 t=1
y = 8.154845485
  by: Separable equation y' = a t^n y  (separate: dy/y = a t^n dt, ln(y/y0) = a t^{n+1}/(n+1))
  a = 2
  n = 1
  y0 = 3
  t = 1
  assumes: n >= 0; t >= 0
```

What the worked example means: This calculation determines the value of y at time t by solving a separable differential equation where the rate of change depends on a power of time. Starting from the initial condition 3, the method integrates the growth factor over time, resulting in an exponential expression involving the coefficient 2 and the power 1. The final computed value is 8.15485, which represents the state of the system after evolving according to these parameters.

Slope (direction) field value — `slope_field`

The slope dy/dt = f(t, y) of the direction field at a point, and its angle.

$$m = f(t, y),\ \theta = \arctan m$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y)	—
`t`	in	t-coordinate	—
`y`	in	y-coordinate	—
`slope`	out	slope f(t, y)	—
`angle_deg`	out	angle of the field line, degrees	—

Applies when: f defined at the point.

```
$ differential-equations solve slope_field "f=t + y" t=1 y=2
slope = 3
angle_deg = 71.56505118
  by: Slope (direction) field value  (evaluate f; angle = atan(slope))
  t = 1
  y = 2
  f = t + y
  assumes: f defined at the point
```

What the worked example means: This calculation determines the local behavior of solution curves for a first-order differential equation at a specific point. Evaluating the right-hand side function f at the given coordinates t and y gives the instantaneous rate of change, which is the slope 3. This value corresponds to an angle of 71.5651 degrees relative to the horizontal axis. These values help visualize how solutions behave near that point without needing to solve the equation analytically.

Time for exponential growth or decay to reach a value — `exponential_time_to_reach`

For dy/dt = k*y, y(0) = y0: the time at which y reaches y1.

$$t = \frac{1}{k} \ln \frac{y_1}{y_0}$$

symbol		meaning	SI unit
`y0`	in	initial value	—
`y1`	in	target value	—
`k`	in	rate constant	—
`t`	out	time to reach y1	—

Applies when: k != 0; y1/y0 > 0; the target lies ahead (t >= 0).

```
$ differential-equations solve exponential_time_to_reach y0=100 k=0.05 y1=300
t = 21.97224577
  by: Time for exponential growth or decay to reach a value  (t = ln(y1/y0)/k)
  y0 = 100
  k = 0.05
  y1 = 300
  assumes: k != 0; y1/y0 > 0; the target lies ahead (t >= 0)
```

What the worked example means: This calculation determines the specific moment when an exponentially changing quantity reaches a desired level. Using the initial value 100, the target value 300, and the rate constant 0.05 gives the elapsed time 21.9722. This result represents the duration required for the system to transition from its starting state to the specified endpoint, assuming the growth or decay follows the model dy/dt = k*y.

Laplace transforms (7)

Convolution integral (f * g)(t) — `convolution_integral`

(f * g)(t) = integral from 0 to t of f(u) g(t - u) du, whose transform is F(s) G(s).

$$(f * g)(t) = \int_0^t f(u)\, g(t-u)\, du$$

symbol		meaning	SI unit
`f`	in	function f(t)	—
`g`	in	function g(t)	—
`t`	in	time (>= 0)	—
`value`	out	(f * g)(t)	—

Applies when: f, g integrable on [0, t].

```
$ differential-equations solve convolution_integral "f=exp(-t)" "g=sin(t)" t=2
value = 0.7303897733
  by: Convolution integral (f * g)(t)  (adaptive tanh-sinh quadrature of the convolution integral
at 20 digits)
  t = 2
  f = exp(-t)
  g = sin(t)
  assumes: f, g integrable on [0, t]
```

What the worked example means: This calculation determines the convolution of two functions, f and g, at a specific time t. The method involves integrating the product of one function shifted by the other over the interval from zero to t. For the inputs provided, where the first function is exp(-t) and the second is sin(t), the resulting value of the convolution integral is 0.73039. This quantity represents the combined effect of the two functions up to that moment in time.

Impulse response of a second-order ODE — `ode_impulse_response`

The response h(t) of a y'' + b y' + c y = delta(t) from rest (the inverse Laplace transform of 1/(a s^2 + b s + c)).

$$h(t) = \mathcal{L}^{-1}\left\{\frac{1}{as^2 + bs + c}\right\}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`t`	in	time (>= 0)	—
`h`	out	impulse response h(t)	—

Applies when: a != 0; t >= 0.

```
$ differential-equations solve ode_impulse_response a=1 b=2 c=5 t=1
h = 0.1672559146
  by: Impulse response of a second-order ODE  (the impulse gives y(0+) = 0, y'(0+) = 1/a; then the
homogeneous solution)
  a = 1
  b = 2
  c = 5
  t = 1
  assumes: a != 0; t >= 0
```

What the worked example means: This calculation determines the impulse response of a second-order linear differential equation. Given the coefficient of the second derivative 1, the coefficient of the first derivative 2, and the constant term 5, the system's reaction to an instantaneous unit input is evaluated at time 1. The resulting value 0.167256 represents the magnitude of the system output at that specific moment, characterizing how energy from the impulse dissipates or oscillates over time.

Inverse Laplace transform of a rational function — `inverse_laplace_transform`

f(t) for a strictly proper rational F(s) (any poles, repeated or complex), by residues / partial fractions.

$$f(t) = \sum_{p} \mathrm{Res}_{s=p}\ F(s)e^{st}$$

symbol		meaning	SI unit
`F`	in	F(s), a rational function of s	—
`t`	in	time (>= 0)	—
`f`	out	value f(t)	—
`poles`	out	poles of F(s) (canonical order)	—

Applies when: F strictly proper with rational coefficients; denominator degree <= 10.

```
$ differential-equations solve inverse_laplace_transform "F=(s + 3)/(s^2 + 3s + 2)" t=1
f = 0.6004235991
poles = [-2.0,-1.0]
  by: Inverse Laplace transform of a rational function  (f(t) = sum over poles p (order m) of the
residue of F(s) e^{st}: e^{pt} sum_k g_k t^{m-1-k}/(m-1-k)!)
  t = 1
  F = (s + 3)/(s^2 + 3s + 2)
  assumes: F strictly proper with rational coefficients; denominator degree <= 10
```

What the worked example means: This calculation determines the time-domain function f(t) from its Laplace transform F(s). By applying residue calculus to the poles of (s + 3)/(s^2 + 3s + 2), the method computes the value 0.600424 at the specific time 1. The result represents the system's response in the time domain, derived from the frequency-domain representation provided by the input function.

Laplace transform of a delayed function u(t - c) f(t - c) — `laplace_time_shift`

Second shifting theorem: the transform of u(t - c) f(t - c) is e^{-cs} F(s).

$$\mathcal{L}\{u(t-c)f(t-c)\} = e^{-cs}F(s)$$

symbol		meaning	SI unit
`f`	in	function f(t)	—
`c`	in	delay (>= 0)	—
`s`	in	transform variable	—
`F`	out	transform of the delayed function	—
`G`	out	transform of f itself	—

Applies when: c >= 0; s > abscissa of f.

```
$ differential-equations solve laplace_time_shift f=t c=2 s=1
F = 0.1353352832
G = 1
  by: Laplace transform of a delayed function u(t - c) f(t - c)  (e^{-cs} times the table transform
of f)
  c = 2
  s = 1
  f = t
  assumes: c >= 0; s > abscissa of f
```

What the worked example means: This calculation applies the second shifting theorem to determine the Laplace transform of a function that begins after a delay. Multiplying the transform of the base function, denoted as 1, by an exponential term involving the delay time 2 and the frequency variable 1 gives the final result 0.135335. This method allows practitioners to handle piecewise continuous inputs or signals that are switched on at a specific time.

Laplace transform of a function at a point — `laplace_transform`

F(s) = integral from 0 to infinity of e^{-st} f(t) dt for sums of t^n e^{at} sin/cos(bt) (also sinh, cosh), and the abscissa of convergence.

$$\mathcal{L}\{t^n e^{zt}\}(s) = \frac{n!}{(s-z)^{n+1}}$$

symbol		meaning	SI unit
`f`	in	function f(t)	—
`s`	in	transform variable (s > abscissa)	—
`F`	out	transform value F(s)	—
`sigma`	out	abscissa of convergence (F exists for s > sigma)	—

Applies when: f a sum of t^n e^{at} sin/cos(bt) terms; s > sigma.

```
$ differential-equations solve laplace_transform "f=t^2 e^(3t)" s=5
F = 0.25
sigma = 3
  by: Laplace transform of a function at a point  (table: L{t^n e^{zt}} = n!/(s - z)^{n+1}, with
sin and cos written through complex exponentials)
  s = 5
  f = t^2 e^(3t)
  assumes: f a sum of t^n e^{at} sin/cos(bt) terms; s > sigma
```

What the worked example means: This calculation determines the Laplace transform, denoted as F(s), for a function composed of polynomial, exponential, and trigonometric terms. By applying the table formula involving factorials and powers of the difference between s and the complex exponent, the integral from zero to infinity is converted into an algebraic expression. The result, 0.25, represents the frequency-domain equivalent of the time-domain signal t^2 e^(3t) at the specific transform variable value 5. The computed abscissa of convergence, 3, indicates that this transform exists and is valid only when s exceeds this threshold.

Partial fractions of a rational function (simple poles) — `partial_fraction_residues`

F(s) = sum r_i/(s - p_i): the poles and residues of a strictly proper rational function with simple poles.

$$F(s) = \sum_i \frac{r_i}{s - p_i}, \quad r_i = \frac{P(p_i)}{Q'(p_i)}$$

symbol		meaning	SI unit
`F`	in	F(s), a rational function of s	—
`poles`	out	poles p_i (real ascending, then complex)	—
`residues`	out	residues r_i in the same order	—
`count`	out	number of poles	—

Applies when: strictly proper; simple (distinct) poles.

```
$ differential-equations solve partial_fraction_residues "F=(s + 3)/(s^2 + 3s + 2)"
poles = [-2.0,-1.0]
residues = [-1.0,2.0]
count = 2
  by: Partial fractions of a rational function (simple poles)  (r_i = P(p_i)/Q'(p_i) (cover-up
rule))
  F = (s + 3)/(s^2 + 3s + 2)
  assumes: strictly proper; simple (distinct) poles
```

What the worked example means: This algorithm breaks down the rational function (s + 3)/(s^2 + 3s + 2) into a sum of simpler fractions. It identifies each pole location and calculates its residue using the cover-up rule. The result is a list of 2 distinct poles shown in [-2, -1], paired with their respective coefficients [-1, 2]. This form is essential for solving differential equations via Laplace transforms.

Unit-step response of a second-order ODE — `ode_step_response`

The response of a y'' + b y' + c y = u(t) (unit step) from rest, and its final value 1/c when stable.

$$y(t) = \mathcal{L}^{-1}\left\{\frac{1}{s(as^2 + bs + c)}\right\}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`t`	in	time (>= 0)	—
`y`	out	step response y(t)	—
`final_value`	out	limit 1/c (stable systems)	—

Applies when: a, c != 0; t >= 0.

```
$ differential-equations solve ode_step_response a=1 b=2 c=5 t=1
y = 0.1971671902
final_value = 0.2
  by: Unit-step response of a second-order ODE  (Y(s) = 1/(s(as^2 + bs + c)): constant 1/c plus the
homogeneous solution with y(0) = -1/c, y'(0) = 0)
  a = 1
  b = 2
  c = 5
  t = 1
  assumes: a, c != 0; t >= 0
```

What the worked example means: This calculation determines the time-domain output of a second-order linear system starting from rest when subjected to a constant unit step input. The resulting response, denoted as 0.197167, describes how the system state evolves over time t = 1. For stable systems, this trajectory converges to a steady-state value of 0.2, which represents the long-term effect of the input scaled by the system's static gain.

Linear systems (4)

Linear 2x2 system: equilibrium type and stability (trace-determinant plane) — `phase_portrait_2x2`

Classify the origin of x' = A x (node, saddle, spiral, center, star, degenerate) and its stability from the trace and determinant.

$$\lambda^2 - T\lambda + D = 0,\ T = \operatorname{tr} A,\ D = \det A$$

symbol		meaning	SI unit
`A`	in	the 2x2 coefficient matrix of x' = A x	—
`type`	out	phase portrait type	—
`stability`	out	stability	—
`lambda1`	out	first eigenvalue	—
`lambda2`	out	second eigenvalue	—
`trace`	out	trace	—
`det`	out	determinant	—

Applies when: A real 2x2.

```
$ differential-equations solve phase_portrait_2x2 "A=[[0, 1], [-2, -3]]"
type = stable node
stability = asymptotically stable
lambda1 = -2
lambda2 = -1
trace = -3
det = 2
  by: Linear 2x2 system: equilibrium type and stability (trace-determinant plane)  (exact trace T,
determinant D and T^2 - 4D: the trace-determinant plane)
  assumes: A real 2x2
```

What the worked example means: This analysis classifies the equilibrium at the origin for the linear system defined by matrix A. Computing the trace -3 and determinant 2 gives the eigenvalues -2 and -1. These values dictate whether trajectories converge to or diverge from the origin, or if they oscillate in a spiral or center pattern, resulting in a phase portrait type of stable node.

Linear system x' = A x: solution at a time — `linear_system_solution`

Solve x' = A x, x(0) = x0 (n up to 6) and give x(t) = e^{At} x0.

$$\mathbf{x}(t) = e^{At}\mathbf{x}_0$$

symbol		meaning	SI unit
`A`	in	n x n coefficient matrix	—
`x0`	in	initial vector x(0)	—
`t`	in	time	—
`x`	out	state x(t)	—
`x1`	out	first component of x(t)	—

Applies when: A square, n <= 6.

```
$ differential-equations solve linear_system_solution "A=[[0, 1], [-2, -3]]" "x0=[1, 0]" t=1
x = [0.600423599106,-0.46508831587]
x1 = 0.6004235991
  by: Linear system x' = A x: solution at a time  (matrix exponential at 30 digits (scaling and
squaring of the Taylor series))
  t = 1
  assumes: A square, n <= 6
```

What the worked example means: This calculation determines the state vector of a linear system at a specific future time. It applies the matrix exponential to the coefficient matrix and initial vector provided as inputs. The resulting state [0.600424, -0.465088] represents the system's configuration, where the first component 0.600424 indicates the value of that specific variable at time 1.

Routh-Hurwitz stability test of a characteristic polynomial — `routh_hurwitz`

How many roots of a polynomial lie in the right half plane (Routh array), hence whether the system is asymptotically stable.

$$\#\{\mathrm{Re}\, s > 0\} = \text{sign changes in the first column}$$

symbol		meaning	SI unit
`coeffs`	in	coefficients, highest power first	—
`stable`	out	all roots in the open left half plane	—
`rhp_roots`	out	number of right-half-plane roots	—
`first_column`	out	first column of the Routh array	—

Applies when: regular Routh array (no zero in the first column).

```
$ differential-equations solve routh_hurwitz "coeffs=[1, 6, 11, 6]"
stable = true
rhp_roots = 0
first_column = [1.0,6.0,10.0,6.0]
  by: Routh-Hurwitz stability test of a characteristic polynomial  (Routh array; sign changes in
the first column)
  assumes: regular Routh array (no zero in the first column)
```

What the worked example means: This calculation constructs a Routh array from the polynomial coefficients provided as input. Examining the signs of the elements in the first column counts the number of sign changes to determine how many roots lie in the right half plane. The result shows that there are 0 such roots, which means the system is yes. If all roots were in the left half plane, the system would be asymptotically stable; otherwise, it exhibits unstable modes.

Stability of x' = A x (n x n) from the characteristic polynomial — `linear_system_stability`

Whether the origin of x' = A x is asymptotically stable and how many eigenvalues have positive real part, without computing eigenvalues.

$$\det(\lambda I - A) = \lambda^n + c_1\lambda^{n-1} + \cdots + c_n$$

symbol		meaning	SI unit
`A`	in	n x n coefficient matrix (n <= 8)	—
`stable`	out	asymptotically stable (all eigenvalues Re < 0)	—
`unstable_eigenvalues`	out	number of eigenvalues with Re > 0	—
`char_poly`	out	characteristic polynomial coefficients	—

Applies when: regular Routh array (no eigenvalue on the imaginary axis).

```
$ differential-equations solve linear_system_stability "A=[[-1, 2, 0], [0, -3, 1], [1, 0, -2]]"
stable = true
unstable_eigenvalues = 0
char_poly = [1.0,6.0,11.0,4.0]
  by: Stability of x' = A x (n x n) from the characteristic polynomial  (characteristic polynomial
by Faddeev-LeVerrier (exact), then the Routh array)
  assumes: regular Routh array (no eigenvalue on the imaginary axis)
```

What the worked example means: This calculation determines the stability of the linear system defined by matrix A without explicitly solving for eigenvalues. Constructing the characteristic polynomial using the Faddeev-LeVerrier algorithm and analyzing its coefficients via the Routh array shows whether all roots lie in the left half-plane. The result indicates that the origin is yes, with exactly 0 eigenvalues exhibiting positive real parts, derived from the computed coefficient sequence [1, 6, 11, 4].

Numerical methods (13)

Backward (implicit) Euler method — `backward_euler_method`

Approximate y' = f(t, y), y(t0) = y0 after n implicit Euler steps: y_{k+1} = y_k + h f(t_{k+1}, y_{k+1}).

$$y_{k+1} = y_k + h f(t_{k+1}, y_{k+1})$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y) of y' = f(t, y)	—
`t0`	in	initial time	—
`y0`	in	initial value y(t0)	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_n`	out	approximation of y(t0 + n h)	—
`t_n`	out	final time t0 + n h	—

Applies when: h > 0; h*|df/dy| < 1 at each step (unique step).

```
$ differential-equations solve backward_euler_method "f=-2*y + t" y0=1 h=0.1 n=2 t0=0
y_n = 0.7180555556
t_n = 0.2
  by: Backward (implicit) Euler method  (each step solved by Newton's method from the explicit
Euler predictor)
  y0 = 1
  h = 0.1
  t0 = 0
  f = -2*y + t
  assumes: h > 0; h*|df/dy| < 1 at each step (unique step)
```

What the worked example means: This calculation determines the approximate state of a system described by a differential equation after a series of implicit steps. It begins with the initial condition 1 at time 0 and advances through 2 steps of size 0.1. The final computed state is 0.718056 at the end time 0.2. Unlike explicit methods, each step requires solving an equation for the future state using Newton's method, which provides stability for stiff problems where rapid changes occur.

CFL condition for the advection equation (upwind scheme) — `cfl_condition`

Courant number C = |c| dt/dx of the explicit upwind scheme for u_t + c u_x = 0, its stability (C <= 1) and the largest stable dt.

$$C = \frac{|c|\,\Delta t}{\Delta x} \le 1$$

symbol		meaning	SI unit
`c`	in	advection speed	—
`dt`	in	time step (> 0)	—
`dx`	in	grid spacing (> 0)	—
`courant`	out	Courant number |c| dt / dx	—
`stable`	out	C <= 1	—
`dt_max`	out	largest stable time step dx / |c|	—

Applies when: c != 0; dt, dx > 0.

```
$ differential-equations solve cfl_condition c=2 dt=0.01 dx=0.05
courant = 0.4
stable = true
dt_max = 0.025
  by: CFL condition for the advection equation (upwind scheme)  (Courant-Friedrichs-Lewy condition
(von Neumann: |1 - C(1 - e^{-i theta})| <= 1))
  c = 2
  dt = 0.01
  dx = 0.05
  assumes: c != 0; dt, dx > 0
```

What the worked example means: This calculation determines the stability of the explicit upwind finite difference scheme for the advection equation by computing the Courant number from the advection speed, time step, and grid spacing. The resulting Courant number of 0.4 indicates whether the chosen time step satisfies the condition that this value must be less than or equal to one. Since the computed state is yes, the simulation parameters are within the safe range, and the largest permissible time step before instability occurs is calculated as 0.025.

Classical Runge-Kutta method (RK4) — `runge_kutta_4`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the classical fourth-order Runge-Kutta method.

$$y_{k+1} = y_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y) of y' = f(t, y)	—
`t0`	in	initial time	—
`y0`	in	initial value y(t0)	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_n`	out	approximation of y(t0 + n h)	—
`t_n`	out	final time t0 + n h	—

Applies when: h > 0; 1 <= n <= 10000; f defined along the steps.

```
$ differential-equations solve runge_kutta_4 "f=t + y" y0=1 h=0.1 n=2 t0=0
y_n = 1.242805142
t_n = 0.2
  by: Classical Runge-Kutta method (RK4)  (classical RK4 with four slope evaluations per step)
  y0 = 1
  h = 0.1
  t0 = 0
  f = t + y
  assumes: h > 0; 1 <= n <= 10000; f defined along the steps
```

What the worked example means: This calculation uses the classical fourth-order Runge-Kutta method to approximate the solution of a differential equation. Starting from an initial value of 1 at time 0, it advances through 2 steps of size 0.1. The resulting estimate for the dependent variable at the end of the interval is 1.24281, which corresponds to the time 0.2. This method combines four slope evaluations per step to achieve high accuracy while balancing computational effort.

Crank-Nicolson steps for the heat equation — `crank_nicolson_steps`

Advance grid values of u_t = alpha u_xx by n Crank-Nicolson steps (unconditionally stable), ends held fixed.

$$-\tfrac{r}{2}u_{i-1}^{k+1} + (1+r)u_i^{k+1} - \tfrac{r}{2}u_{i+1}^{k+1} = \tfrac{r}{2}u_{i-1}^{k} + (1-r)u_i^{k} + \tfrac{r}{2}u_{i+1}^{k}$$

symbol		meaning	SI unit
`u`	in	grid values (first and last are boundary values)	—
`r`	in	mesh ratio alpha dt / dx^2 (> 0)	—
`n`	in	number of time steps	—
`u`	out	grid values after n steps	—

Applies when: Dirichlet ends; r > 0.

```
$ differential-equations solve crank_nicolson_steps "u=[0, 50, 100, 50, 0]" r=1 n=1
u = [0.0,35.7142857143,42.8571428571,35.7142857143,0.0]
  by: Crank-Nicolson steps for the heat equation  (average of explicit and implicit steps; each
step a tridiagonal solve (Thomas algorithm))
  r = 1
  assumes: Dirichlet ends; r > 0
```

What the worked example means: This calculation determines the temperature distribution along a rod after a specific number of time intervals. It takes the starting grid values [0, 50, 100, 50, 0], applies the Crank-Nicolson method for 1 steps using the stability parameter 1, and produces the final grid values [0, 35.7143, 42.8571, 35.7143, 0]. The method averages explicit and implicit finite difference schemes to ensure numerical stability regardless of the step size, making it suitable for long-term simulations of heat diffusion.

Euler's method — `euler_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with Euler's method.

$$y_{k+1} = y_k + h f(t_k, y_k)$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y) of y' = f(t, y)	—
`t0`	in	initial time	—
`y0`	in	initial value y(t0)	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_n`	out	approximation of y(t0 + n h)	—

symbol		meaning	SI unit
`t_n`	out	final time t0 + n h	—

Applies when: h > 0; 1 <= n <= 10000; f defined along the steps.

```
$ differential-equations solve euler_method "f=t + y" y0=1 h=0.1 n=2 t0=0
y_n = 1.22
t_n = 0.2
  by: Euler's method  (explicit Euler: y_{k+1} = y_k + h f(t_k, y_k))
  y0 = 1
  h = 0.1
  t0 = 0
  f = t + y
  assumes: h > 0; 1 <= n <= 10000; f defined along the steps
```

What the worked example means: Euler's method computes an approximate solution to the differential equation defined by the right-hand side function. Starting from the initial value 1 at time 0, the algorithm advances the solution in discrete steps of size 0.1. After performing 2 such steps, the final time reached is 0.2, and the corresponding estimated value of the dependent variable is 1.22. This result represents a tangent-line approximation that accumulates error over each step but provides a straightforward numerical estimate for the system's state.

Euler's method error for exponential growth — `euler_error_exponential`

For y' = k y, y(0) = y0: the Euler approximation after n steps of size h, the exact value and the global error.

$$y_n = y_0(1 + hk)^n,\ y(t_n) = y_0 e^{k t_n}$$

symbol		meaning	SI unit
`k`	in	rate constant	—
`y0`	in	initial value	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_euler`	out	Euler value y0 (1 + h k)^n	—
`y_exact`	out	exact value y0 e^{k n h}	—
`error`	out	global error exact - Euler	—
`relative_error`	out	error / exact	—

Applies when: h > 0.

```
$ differential-equations solve euler_error_exponential k=1 y0=1 h=0.1 n=10
y_euler = 2.59374246
y_exact = 2.718281828
error = 0.1245393684
relative_error = 0.04581547324
  by: Euler's method error for exponential growth  (Euler on a linear equation multiplies by (1 +
hk) per step)
  k = 1
  y0 = 1
  h = 0.1
  assumes: h > 0
```

What the worked example means: This calculation determines the numerical solution to an exponential growth equation using Euler's method. Given a rate constant of 1 and an initial value of 1, the method advances the solution by a step size of 0.1 for 10 steps. The resulting approximate value is 2.59374, which is compared to the exact analytical result of 2.71828. The difference between these values, known as the global error, is 0.124539, while the relative error is 0.0458155. This comparison quantifies how much the discrete stepping process deviates from the true continuous curve.

Explicit finite-difference (FTCS) steps for the heat equation — `heat_ftcs_steps`

Advance grid values of u_t = alpha u_xx by n explicit steps u_i <- u_i + r (u_{i+1} - 2 u_i + u_{i-1}), ends held fixed.

$$u_i^{k+1} = u_i^k + r\left(u_{i+1}^k - 2u_i^k + u_{i-1}^k\right)$$

symbol		meaning	SI unit
`u`	in	grid values (first and last are boundary values)	—
`r`	in	mesh ratio alpha dt / dx^2 (> 0)	—
`n`	in	number of time steps	—
`u`	out	grid values after n steps	—
`stable_ratio`	out	r <= 1/2	—

Applies when: Dirichlet ends (held fixed); r > 0.

```
$ differential-equations solve heat_ftcs_steps "u=[0, 50, 100, 50, 0]" r=0.25 n=1
u = [0.0,50.0,75.0,50.0,0.0]
stable_ratio = true
  by: Explicit finite-difference (FTCS) steps for the heat equation  (forward time, centred space)
  r = 0.25
  assumes: Dirichlet ends (held fixed); r > 0
```

What the worked example means: This calculation advances the solution of the heat equation using the explicit forward-time centered-space method. Starting from initial grid values [0, 50, 100, 50, 0], it applies 1 time steps with a mesh ratio of 0.25. The resulting grid values [0, 50, 75, 50, 0] represent the temperature distribution at the final time step, provided the stability condition yes is met.

Explicit midpoint method (RK2) — `midpoint_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the explicit midpoint method.

$$y_{k+1} = y_k + hf\left(t_k + \tfrac{h}{2}, y_k + \tfrac{h}{2} f(t_k, y_k)\right)$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y) of y' = f(t, y)	—
`t0`	in	initial time	—
`y0`	in	initial value y(t0)	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_n`	out	approximation of y(t0 + n h)	—
`t_n`	out	final time t0 + n h	—

Applies when: h > 0; 1 <= n <= 10000; f defined along the steps.

```
$ differential-equations solve midpoint_method "f=t + y" y0=1 h=0.1 n=2 t0=0
y_n = 1.24205
t_n = 0.2
  by: Explicit midpoint method (RK2)  (second-order Runge-Kutta, slope at the midpoint)
  y0 = 1
  h = 0.1
  t0 = 0
  f = t + y
  assumes: h > 0; 1 <= n <= 10000; f defined along the steps
```

What the worked example means: This calculation determines an approximation for the solution of a differential equation using the explicit midpoint method. Starting from the initial condition defined by time 0 and value 1, the algorithm advances through n steps of size h. The resulting estimate for the dependent variable is 1.24205, which represents the solution at the computed final time 0.2. This method improves upon basic Euler approaches by evaluating the slope at the midpoint of each interval, providing a more accurate representation of the curve's behavior over the step.

Improved Euler (Heun) method — `improved_euler_method`

Approximate the solution of y' = f(t, y), y(t0) = y0, after n steps of size h with the improved Euler (Heun) method.

$$y_{k+1} = y_k + \tfrac{h}{2}\left[f(t_k, y_k) + f(t_{k+1}, y_k + hf(t_k, y_k))\right]$$

symbol		meaning	SI unit
`f`	in	right-hand side f(t, y) of y' = f(t, y)	—
`t0`	in	initial time	—
`y0`	in	initial value y(t0)	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`y_n`	out	approximation of y(t0 + n h)	—

symbol		meaning	SI unit
`t_n`	out	final time t0 + n h	—

Applies when: h > 0; 1 <= n <= 10000; f defined along the steps.

```
$ differential-equations solve improved_euler_method "f=t + y" y0=1 h=0.1 n=2 t0=0
y_n = 1.24205
t_n = 0.2
  by: Improved Euler (Heun) method  (Heun predictor-corrector: average the slopes at both ends)
  y0 = 1
  h = 0.1
  t0 = 0
  f = t + y
  assumes: h > 0; 1 <= n <= 10000; f defined along the steps
```

What the worked example means: Using the Improved Euler method approximates the solution to the differential equation by averaging slopes. Starting from initial value 1 at time 0, after taking steps of size 0.1 for a total of 2 iterations, the estimated function value reaches 1.24205 at the final time 0.2. This result reflects how the system evolves under the given derivative rule.

Largest stable step of an explicit method for y' = lambda y — `explicit_step_limit`

The absolute-stability step limit h_max of Euler, Heun, midpoint or RK4 on a decaying mode y' = lambda y (lambda < 0).

$$|R(h\lambda)| \le 1: \; h \le \frac{x^*}{|\lambda|} \; (x^* = 2\,\text{Euler}, 2.785\,\text{RK4})$$

symbol		meaning	SI unit
`method`	in	explicit method	—
`lam`	in	lambda (< 0)	—
`h_max`	out	largest stable step	—
`z_boundary`	out	real stability boundary h lambda	—

Applies when: lambda < 0.

```
$ differential-equations solve explicit_step_limit lam=-20 method=rk4
h_max = 0.1392646782
z_boundary = -2.785293563
  by: Largest stable step of an explicit method for y' = lambda y  (bisection for |R(-x)| = 1 on
the stability polynomial)
  lam = -20
  method = rk4
  assumes: lambda < 0
```

What the worked example means: This calculation determines the maximum time step size, 0.139265, that an explicit numerical method can use while maintaining absolute stability when solving a decaying linear differential equation. The result is derived from the specific stability polynomial of the selected method and the decay rate -20. The intermediate value -2.78529 represents the product of this maximum step and the decay rate, marking the point where the magnitude of the stability function equals one. For standard methods

like Euler or RK4, this boundary lies on the negative real axis, ensuring that the numerical solution does not grow unbounded despite the physical decay.

RK4 for a system of two first-order equations — `runge_kutta_4_system`

Approximate x' = f(t, x, y), y' = g(t, x, y) with x(t0) = x0, y(t0) = y0 after n RK4 steps of size h.

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \tfrac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

symbol		meaning	SI unit
`f`	in	x' = f(t, x, y)	—
`g`	in	y' = g(t, x, y)	—
`t0`	in	initial time	—
`x0`	in	initial x	—
`y0`	in	initial y	—
`h`	in	step size (> 0)	—
`n`	in	number of steps	—
`x_n`	out	approximation of x(t0 + n h)	—
`y_n`	out	approximation of y(t0 + n h)	—
`t_n`	out	final time	—

Applies when: h > 0.

```
$ differential-equations solve runge_kutta_4_system f=y g=-x x0=1 y0=0 t0=0 h=0.1 n=10
x_n = 0.5403029671
y_n = -0.8414704778
t_n = 1
  by: RK4 for a system of two first-order equations  (classical RK4 applied to the vector (x, y))
  x0 = 1
  y0 = 0
  t0 = 0
  h = 0.1
  g = -x
  f = y
  assumes: h > 0
```

What the worked example means: This calculation uses the fourth-order Runge-Kutta method to approximate the solution of a system of two coupled first-order differential equations. Starting from initial values at time 0, it advances through n steps of size 0.1 to compute the state variables x and y. The resulting approximation for x at the final time is 0.540303, and for y it is -0.84147, representing the estimated trajectory of the system.

Stability function (amplification factor) of a one-step method — `amplification_factor`

For y' = lambda y: the factor R(h lambda) a method multiplies by per step, its size, and whether the step is stable (|R| <= 1).

$$y_{k+1} = R(h\lambda)\, y_k$$

symbol		meaning	SI unit
`method`	in	the one-step method	—
`lam`	in	lambda (real)	—
`h`	in	step size (> 0)	—
`R`	out	amplification factor R(h lambda)	—
`abs_R`	out	|R|	—
`stable`	out	|R| <= 1	—
`z`	out	h lambda	—

Applies when: h > 0; real lambda.

```
$ differential-equations solve amplification_factor lam=-10 h=0.2 method=rk4
R = 0.3333333333
abs_R = 0.3333333333
stable = true
z = -2
  by: Stability function (amplification factor) of a one-step method  (R(z): 1 + z (Euler), 1 + z +
z^2/2 (Heun, midpoint), Taylor to z^4/24 (RK4), 1/(1 - z), (1 + z/2)/(1 - z/2))
  lam = -10
  h = 0.2
  method = rk4
  assumes: h > 0; real lambda
```

What the worked example means: This calculation determines the stability function value for the specified one-step method applied to the linear test equation. Substituting the product of the step size and the rate constant into the method's specific polynomial or rational form gives the amplification factor 0.333333. The absolute value 0.333333 indicates whether the numerical solution grows or decays per step, while the boolean flag yes confirms if the step remains within the stability region where the magnitude does not exceed unity.

Stability of the explicit (FTCS) scheme for the heat equation — `explicit_heat_stability`

The mesh ratio r = alpha dt / dx^2 of the explicit scheme for u_t = alpha u_xx, its stability (r <= 1/2) and the largest stable dt.

$$r = \frac{\alpha\,\Delta t}{\Delta x^2} \le \frac{1}{2}$$

symbol		meaning	SI unit
`alpha`	in	diffusivity (> 0)	—
`dt`	in	time step (> 0)	—
`dx`	in	grid spacing (> 0)	—
`r`	out	mesh ratio alpha dt / dx^2	—
`stable`	out	r <= 1/2	—
`dt_max`	out	largest stable time step dx^2/(2 alpha)	—

Applies when: alpha, dt, dx > 0.

```
$ differential-equations solve explicit_heat_stability alpha=1 dt=0.001 dx=0.05
r = 0.4
stable = true
dt_max = 0.00125
  by: Stability of the explicit (FTCS) scheme for the heat equation  (von Neumann: G = 1 - 4 r
sin^2(theta/2), |G| <= 1 iff r <= 1/2)
  alpha = 1
  dt = 0.001
  dx = 0.05
  assumes: alpha, dt, dx > 0
```

What the worked example means: This calculation determines the stability of the explicit finite difference scheme for the heat equation. It computes the mesh ratio r from the diffusivity, time step, and grid spacing to check if it satisfies the condition that r must be less than or equal to one half. The resulting value of 0.4 indicates whether the numerical solution will remain bounded, while 0.00125 represents the largest time step allowed before instability occurs.

Second-order ODES (12)

Cauchy-Euler (equidimensional) equation: initial value problem — `cauchy_euler_ivp`

Solve a t^2 y'' + b t y' + c y = 0 (t > 0) with y(1) = y1, y'(1) = v1 and give y(t) and the indicial roots.

$$a\,m(m-1)+b\,m+c=0, \quad y=t^m$$

symbol		meaning	SI unit
`a`	in	coefficient of t^2 y''	—
`b`	in	coefficient of t y'	—
`c`	in	coefficient of y	—
`y1`	in	value y(1)	—
`v1`	in	slope y'(1)	—
`t`	in	time (> 0)	—
`y`	out	value y(t)	—
`m1`	out	first indicial root	—
`m2`	out	second indicial root	—

Applies when: a != 0; t > 0.

```
$ differential-equations solve cauchy_euler_ivp a=1 b=2 c=-6 y1=1 v1=0 t=2
y = 2.45
m1 = -3
m2 = 2
  by: Cauchy-Euler (equidimensional) equation: initial value problem  (t = e^x turns it into a Y''
+ (b - a) Y' + c Y = 0; indicial a m(m-1) + b m + c = 0)
  a = 1
  b = 2
  c = -6
  y1 = 1
  v1 = 0
  t = 2
  assumes: a != 0; t > 0
```

What the worked example means: This calculation determines the specific solution for a homogeneous Cauchy-Euler differential equation defined by coefficients 1, 2, and -6. By applying the substitution method, it finds the indicial roots -3 and 2, which characterize the exponential behavior of the system. Using these roots along with the initial conditions 1 for position and 0 for velocity at time one, the model computes the exact state 2.45 at the specified time 2. This result represents the trajectory of the system under the given constraints.

Damped oscillator: resonance frequency and peak amplitude — `practical_resonance`

For m y'' + c y' + k y = F0 cos(w t): the forcing frequency of maximum steady-state amplitude and that amplitude.

$$\omega_r = \sqrt{\frac{k}{m} - \frac{c^2}{2m^2}}, \quad A_{\max} = \frac{F_0}{c\sqrt{k/m - c^2/(4m^2)}}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`c`	in	damping coefficient (> 0)	—
`k`	in	spring constant (> 0)	—
`F0`	in	forcing amplitude	—
`omega_r`	out	resonance (peak) frequency, 0 when there is no peak	—
`amplitude_max`	out	peak amplitude	—
`has_peak`	out	c^2 < 2 m k	—

Applies when: m, c, k > 0.

```
$ differential-equations solve practical_resonance m=1 c=0.4 k=4 F0=2
omega_r = 1.979898987
amplitude_max = 2.512594538
has_peak = true
  by: Damped oscillator: resonance frequency and peak amplitude  (minimise (k - m w^2)^2 + (c w)^2
over w: w_r^2 = k/m - c^2/(2m^2))
  m = 1
  c = 0.4
  k = 4
  F0 = 2
  assumes: m, c, k > 0
```

What the worked example means: This calculation determines the forcing frequency that produces the largest steady-state vibration in a damped oscillator. Using the mass, damping coefficient, spring constant, and forcing amplitude, it finds that the resonance occurs at 1.9799, where the system responds with a maximum amplitude of 2.51259. This point represents the optimal energy transfer from the external force to the mechanical system.

Forced damped oscillator: steady-state amplitude and phase — `steady_state_amplitude`

Steady-state response of a y'' + b y' + c y = F0 cos(w t): amplitude, phase lag and gain.

$$A = \frac{F_0}{\sqrt{(c - a\omega^2)^2 + (b\omega)^2}},\ \tan\delta = \frac{b\omega}{c - a\omega^2}$$

symbol		meaning	SI unit
`a`	in	mass-like coefficient (> 0)	—
`b`	in	damping (> 0)	—
`c`	in	stiffness (> 0)	—
`F0`	in	forcing amplitude	—
`w`	in	forcing angular frequency (> 0)	—
`amplitude`	out	steady-state amplitude	—
`phase_lag`	out	phase lag in radians (0 to pi)	—
`gain`	out	amplitude per unit force	—

Applies when: a, b, c > 0; w > 0.

```
$ differential-equations solve steady_state_amplitude a=1 b=0.5 c=4 F0=2 w=1.5
amplitude = 1.050451463
phase_lag = 0.4048917863
gain = 0.5252257314
  by: Forced damped oscillator: steady-state amplitude and phase  (|F0| / sqrt((c - a w^2)^2 + (b
w)^2), lag atan2(b w, c - a w^2))
  a = 1
  b = 0.5
  c = 4
  F0 = 2
  w = 1.5
  assumes: a, b, c > 0; w > 0
```

What the worked example means: This calculation determines the long-term response of a damped oscillator to a sinusoidal force. Given mass-like coefficient 1, damping 0.5, stiffness 4, forcing amplitude 2, and angular frequency 1.5, it computes the resulting steady-state amplitude as 1.05045 and the phase lag as 0.404892. The gain, representing amplitude per unit force, is 0.525226. These values describe how much the system oscillates and by how much it trails behind the driving force once initial transients have faded.

Linear two-point boundary value problem (constant coefficients) — `boundary_value_problem`

Solve y'' + p y' + q y = r on [xa, xb] with y(xa) = ya, y(xb) = yb; give y(x) and the slope at xa.

$$y'' + py' + qy = r,\ y(x_a) = y_a,\ y(x_b) = y_b$$

symbol		meaning	SI unit
`p`	in	coefficient of y'	—
`q`	in	coefficient of y	—
`r`	in	constant right-hand side	—
`xa`	in	left end	—
`xb`	in	right end (> xa)	—
`ya`	in	value at xa	—
`yb`	in	value at xb	—
`x`	in	point in [xa, xb]	—
`y`	out	value y(x)	—
`slope_a`	out	slope y'(xa)	—

Applies when: xa < xb; unique solution (not an eigenvalue case).

```
$ differential-equations solve boundary_value_problem p=0 q=-1 r=0 xa=0 xb=1 ya=0 yb=1 x=0.5
y = 0.443409442
slope_a = 0.8509181282
  by: Linear two-point boundary value problem (constant coefficients)  (particular + homogeneous
basis; boundary conditions fix the two constants)
  p = 0
  q = -1
  r = 0
  xa = 0
  xb = 1
  ya = 0
  yb = 1
  x = 0.5
  assumes: xa < xb; unique solution (not an eigenvalue case)
```

What the worked example means: This calculation determines the unique function satisfying the linear boundary value problem across the interval. It combines a particular solution for the non-homogeneous term with homogeneous basis functions to meet the endpoint constraints. The resulting curve passes through the fixed boundaries, yielding a specific height of 0.443409 at the evaluation location and an initial gradient of 0.850918 at the left edge.

Mass-spring-damper: natural frequency, damping ratio, damped frequency — `damped_oscillator_parameters`

For m x'' + c x' + k x = 0: natural frequency, damping ratio, damping case and (underdamped) damped frequency and period.

$$\omega_0 = \sqrt{k/m},\ \zeta = \frac{c}{2\sqrt{mk}},\ \omega_d = \omega_0\sqrt{1-\zeta^2}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`c`	in	damping coefficient (>= 0)	—
`k`	in	spring constant (> 0)	—
`omega0`	out	natural frequency sqrt(k/m)	—
`zeta`	out	damping ratio c/(2 sqrt(mk))	—
`damping`	out	damping case	—
`omega_d`	out	damped frequency (underdamped)	—
`period_d`	out	damped period (underdamped)	—

Applies when: m > 0; c >= 0; k > 0.

```
$ differential-equations solve damped_oscillator_parameters m=2 c=1 k=8
omega0 = 2
zeta = 0.125
damping = underdamped
omega_d = 1.984313483
period_d = 3.166427765
  by: Mass-spring-damper: natural frequency, damping ratio, damped frequency  (w0 = sqrt(k/m), zeta
= c/(2 sqrt(mk)), w_d = w0 sqrt(1 - zeta^2))
  m = 2
  c = 1
  k = 8
  assumes: m > 0; c >= 0; k > 0
```

What the worked example means: This calculation determines the dynamic characteristics of a mass-spring-damper system using the given mass, damping coefficient, and spring constant. The undamped natural frequency, calculated as the square root of the stiffness divided by the mass, is 2. The damping ratio, which compares the actual damping to the critical damping value, is 0.125. Based on this ratio, the system is classified as underdamped. If the system is underdamped, the damped natural frequency is 1.98431 and the corresponding period is 3.16643.

Second-order constant-coefficient ODE: characteristic roots — `characteristic_roots`

Roots of the characteristic equation of a y'' + b y' + c y = 0, the form of the general solution and the damping case.

$$ar^2 + br + c = 0, \quad r = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`r1`	out	first root (real ascending; complex: positive imaginary part first)	—
`r2`	out	second root	—
`discriminant`	out	b^2 - 4ac	—
`roots_type`	out	distinct real / repeated / complex	—
`damping`	out	overdamped / critically damped / underdamped / undamped	—
`solution_form`	out	form of the general solution	—

Applies when: a != 0.

```
$ differential-equations solve characteristic_roots a=1 b=3 c=2
r1 = -2
r2 = -1
discriminant = 1
roots_type = two distinct real roots
damping = overdamped
solution_form = y = C1 e^(r1 t) + C2 e^(r2 t)
  by: Second-order constant-coefficient ODE: characteristic roots  (a r^2 + b r + c = 0 (stable
quadratic formula); case from the exact sign of b^2 - 4ac)
  a = 1
  b = 3
  c = 2
  assumes: a != 0
```

What the worked example means: This calculation determines the roots of the characteristic equation for a second-order linear homogeneous differential equation with constant coefficients. Using the input coefficients 1, 3, and 2, the method computes the discriminant 1 to classify the system's response. The resulting roots, -2 and -1, define the form of the general solution as y = C1 e^(r1 t) + C2 e^(r2 t). Based on these values, the physical system is identified as overdamped, which describes how the motion or state evolves over time without external forcing.

Second-order homogeneous linear ODE: initial value problem — `second_order_homogeneous_ivp`

Solve a y'' + b y' + c y = 0 with y(0) = y0, y'(0) = v0 and give y(t) (overdamped, critical or underdamped).

$$y(t) = e^{mt}\left[y_0\cosh(\delta t) + (v_0 - m y_0)\frac{\sinh(\delta t)}{\delta}\right],\ m = -\frac{b}{2a},\ \delta^2 = m^2 - \frac{c}{a}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`y0`	in	initial value y(0)	—
`v0`	in	initial slope y'(0)	—

symbol		meaning	SI unit
`t`	in	time	—
`y`	out	value y(t)	—
`damping`	out	damping case	—

Applies when: a != 0.

```
$ differential-equations solve second_order_homogeneous_ivp a=1 b=3 c=2 y0=1 v0=0 t=1
y = 0.6004235991
damping = overdamped
  by: Second-order homogeneous linear ODE: initial value problem  (y = e^{mt}[y0 C(t) + (v0 - m y0) S(t)], m = -b/(2a), C, S = cosh/sinh, 1/t or cos/sin by the sign of the discriminant)
  a = 1
  b = 3
  c = 2
  y0 = 1
  v0 = 0
  t = 1
  assumes: a != 0
```

What the worked example means: This calculation determines the value of the dependent variable y at a specific time t, given the initial position y0 and initial velocity v0. The result 0.600424 reflects the system's evolution under the coefficients a, b, and c. The behavior is categorized as overdamped, indicating whether the system returns to equilibrium without oscillation, with oscillation, or in a critical state.

Simple harmonic motion: position, amplitude, period — `simple_harmonic_motion`

Solve m x'' + k x = 0 with x(0) = x0, x'(0) = v0: position at t, amplitude, angular frequency and period.

$$x(t) = x_0 \cos \omega t + \frac{v_0}{\omega} \sin \omega t,\ A = \sqrt{x_0^2 + (v_0/\omega)^2}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`k`	in	spring constant (> 0)	—
`x0`	in	initial position	—
`v0`	in	initial velocity	—
`t`	in	time	—
`x`	out	position x(t)	—
`amplitude`	out	amplitude	—
`omega`	out	angular frequency	—
`period`	out	period	—

Applies when: m, k > 0.

```
$ differential-equations solve simple_harmonic_motion m=0.5 k=8 x0=0.1 v0=0 t=1
x = -0.06536436209
amplitude = 0.1
omega = 4
period = 1.570796327
  by: Simple harmonic motion: position, amplitude, period  (x = x0 cos(wt) + (v0/w) sin(wt), w =
sqrt(k/m))
  m = 0.5
  k = 8
  x0 = 0.1
  v0 = 0
  t = 1
  assumes: m, k > 0
```

What the worked example means: This calculation determines the motion of a simple harmonic oscillator using its mass, spring constant, initial position, and initial velocity. The angular frequency is found to be 4, which leads to a period of 1.5708. At time 1, the position is -0.0653644, while the total energy corresponds to an amplitude of 0.1. These values describe how far the object moves from equilibrium and how quickly it completes one cycle.

Sinusoidally forced linear oscillator: initial value problem — `forced_oscillator_ivp`

Solve a y'' + b y' + c y = F0 cos(w t), y(0) = y0, y'(0) = v0 (including pure resonance) and give y(t).

$$y_p = \mathrm{Re}\,\frac{F_0 e^{i\omega t}}{c - a\omega^2 + ib\omega}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`F0`	in	forcing amplitude	—
`w`	in	forcing angular frequency (> 0)	—
`y0`	in	initial value	—
`v0`	in	initial slope	—
`t`	in	time	—
`y`	out	value y(t)	—
`resonant`	out	undamped forcing at the natural frequency	—

Applies when: a != 0; w > 0.

```
$ differential-equations solve forced_oscillator_ivp a=1 b=0 c=4 F0=1 w=2 y0=0 v0=0 t=3
y = -0.2095616236
resonant = true
  by: Sinusoidally forced linear oscillator: initial value problem  (complex amplitude Z = F0/(c -
a w^2 + i b w); resonance: y_p = F0/(2aw) t sin(wt); plus homogeneous part)
  a = 1
  b = 0
  c = 4
  F0 = 1
  w = 2
  y0 = 0
  v0 = 0
  t = 3
  assumes: a != 0; w > 0
```

What the worked example means: This calculation determines the complete time-dependent displacement y of a linear oscillator driven by a sinusoidal force. The result, -0.209562, reflects the balance between the inertial term scaled by coefficient 1, the damping effect from coefficient 0, and the restoring force governed by coefficient 4. The forcing amplitude 1 and angular frequency 2 drive the steady-state component, while initial conditions 0 and 0 shape the transient homogeneous part. If the output flag for resonance is active, it signifies that undamped forcing at the natural frequency causes the amplitude to grow linearly with time t.

Undamped resonance: growing response — `undamped_resonance`

m y'' + k y = F0 cos(w0 t) forced exactly at the natural frequency, starting from rest: y(t) and its linearly growing envelope.

$$y(t) = \frac{F_0}{2m\omega_0}\, t \sin(\omega_0 t), \quad \omega_0 = \sqrt{k/m}$$

symbol		meaning	SI unit
`m`	in	mass (> 0)	—
`k`	in	spring constant (> 0)	—
`F0`	in	forcing amplitude	—
`t`	in	time	—
`y`	out	displacement y(t)	—
`envelope`	out	amplitude envelope F0 t / (2 m w0)	—
`omega0`	out	natural frequency	—

Applies when: m, k > 0; y(0) = y'(0) = 0.

```
$ differential-equations solve undamped_resonance m=1 k=4 F0=2 t=3
y = -0.4191232473
envelope = 1.5
omega0 = 2
  by: Undamped resonance: growing response  (particular solution t sin(w0 t)/(2 m w0) (secular
term))
  m = 1
  k = 4
  F0 = 2
  t = 3
  assumes: m, k > 0; y(0) = y'(0) = 0
```

What the worked example means: This calculation determines the response of an undamped oscillator forced at its natural frequency. Starting from rest, the system accumulates energy without dissipation, causing the amplitude to grow linearly over time. The resulting displacement is -0.419123, while the peak amplitude follows an envelope of 1.5. This behavior highlights the instability of resonant systems when damping is absent.

Undetermined coefficients: exponential forcing — `undetermined_coefficients_exponential`

Solve a y'' + b y' + c y = F e^{s t}, y(0) = y0, y'(0) = v0: the particular coefficient (with resonance) and y(t).

$$y_p = \frac{F}{as^2 + bs + c}\, e^{st}$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`F`	in	forcing amplitude	—
`s`	in	forcing exponent	—
`y0`	in	initial value	—
`v0`	in	initial slope	—
`t`	in	time	—
`y`	out	value y(t)	—
`A`	out	coefficient of the particular solution A t^m e^{st}	—
`resonance_multiplicity`	out	m: 0, or 1/2 when s is a simple/double characteristic root	—

Applies when: a != 0.

```
$ differential-equations solve undetermined_coefficients_exponential a=1 b=3 c=2 F=4 s=3 y0=0 v0=0
t=1
y = 3.75749617
A = 0.2
resonance_multiplicity = 0
  by: Undetermined coefficients: exponential forcing  (y_p = A t^m e^{st}, A = F/P(s), F/P'(s) or
F/(2a); plus the homogeneous solution fitted to the initial values)
  a = 1
  b = 3
  c = 2
  F = 4
  s = 3
  y0 = 0
  v0 = 0
  t = 1
  assumes: a != 0
```

In the worked example, a = 1, b = 3, c = 2, F = 4, s = 3, y0 = 0, v0 = 0, t = 1 go through the equation above and give y = 3.75749617, A = 0.2, resonance_multiplicity = 0. It applies only when a != 0: outside that it says what is wrong instead of returning a number.

Undetermined coefficients: polynomial forcing — `undetermined_coefficients_polynomial`

Solve a y'' + b y' + c y = p0 + p1 t + ... (degree <= 4), y(0) = y0, y'(0) = v0: particular polynomial and y(t).

$$y_p = \sum_k q_k t^k, \quad c\,q_k + b(k+1)q_{k+1} + a(k+2)(k+1)q_{k+2} = p_k$$

symbol		meaning	SI unit
`a`	in	coefficient of y''	—
`b`	in	coefficient of y'	—
`c`	in	coefficient of y	—
`p`	in	forcing coefficients p0, p1, ... (ascending powers of t)	—
`y0`	in	initial value	—
`v0`	in	initial slope	—
`t`	in	time	—
`y`	out	value y(t)	—
`particular`	out	coefficients q0, q1, ... of the particular polynomial	—

Applies when: a != 0; c != 0; degree <= 4.

```
$ differential-equations solve undetermined_coefficients_polynomial "p=[0, 8]" a=1 b=0 c=4 y0=1
v0=0 t=0.5
y = 0.6988313211
particular = [0.0,2.0]
  by: Undetermined coefficients: polynomial forcing  (match powers of t from the top: c q_k + b
(k+1) q_{k+1} + a (k+2)(k+1) q_{k+2} = p_k)
  a = 1
  b = 0
  c = 4
  y0 = 1
  v0 = 0
  t = 0.5
  assumes: a != 0; c != 0; degree <= 4
```

What the worked example means: This calculation determines the particular polynomial solution for a linear differential equation with constant coefficients driven by a polynomial forcing term. By matching powers of time, it solves for the coefficients q0 through q4 that satisfy the equation. The resulting general solution combines this particular part with the homogeneous solution to meet initial conditions. For the given inputs, the computed value of y at time t is 0.698831, derived from the coefficient set [0, 2]. This method assumes the leading coefficient a and the constant term c are non-zero, ensuring the system can be solved uniquely for these specific polynomial degrees.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.